

Build a Reachy robot

The student build book: print it, set up the motors, build it with the Pi, install the software, test it — then make it yours.



Start here	What you're building, the six stages, teams, safety, day one
Stage 1 · Print	Test prints first, then every part — arms included
Stage 2 · Motor IDs	Set up the Pi, give all 17 motors their own ID
Stage 3 · Assembly	Head, antennae, face, arms, base and power — with the Pi
Stage 4 · Software	Name, calibration, arms, autostart, the phone face
Stage 5 · Testing	Acceptance test, the Code Lab (Blockly + JavaScript), VR
Stage 6 · Make it yours	Optional: a different phone, voice and eyes
Stage 6 · Your own head	Design your robot's head in Blender
Appendix A	Shopping list
Appendix B	Mentor notes: kit, SD cards, network, power wiring

Build a Reachy robot — start here

Six teams, six robots. Each team builds a complete **Reachy Mini-style robot**: a head that nods, tilts and turns on six motors, two springy antennae, two arms, a body that turns on a printed base, a phone for a face, and a Raspberry Pi for a brain. Then you make it yours: its own name, its own voice, and if you like, its own head designed in Blender.

They are copies of **Ollie**. Everything in this book has been built and broken once already, and the notes are what that taught.



Ollie: the phone face in the head, the wire antennae, both arms fixed to the top of the body, the base that turns, and the Raspberry Pi in its white case beside the base.

Read this	When
This page	Day one: what you're building, the six stages, teams, safety
Stage 1 — Print	Every printed part, test prints first
Stage 2 — Motor IDs	Set up the Pi, give all 17 motors their own ID
Stage 3 — Assembly	Put the robot together, with the Pi beside you
Stage 4 — Software	Name, calibration, arms, autostart, the phone face
Stage 5 — Testing	The acceptance test, the Code Lab (Blockly and JavaScript), VR control

Read this	When
Stage 6 — Make it yours	Optional: a different phone, voice and eyes, your own head in Blender
Appendix A — Shopping list	What to buy, per robot and for six
Appendix B — Mentor notes	Mentors: the kit, SD cards, the network, the power wiring

What you are building

Part of the robot	What it is	Motors
Head	A 6-motor <i>Stewart platform</i> : six motor arms push six rods that hold up the head. Moving them together makes it nod, tilt, turn and bob	6 × XL330-M288 (IDs 4–9)
Antennae	Two bent-wire antennae on the back of the head — the robot's main way to show emotion	2 × XL330-M077 (IDs 1–2)
Arms	Two 4-joint printed arms driven by belts: shoulder pitch, shoulder roll, elbow, wrist	8 × XL330-M288 (right 14–17, left 18–21)
Body	The "egg" that holds the head motors. The arms screw to the top of it	—
Base	A printed plinth with a bearing on top; the whole robot turns on it	1 × XC330-M288 (ID 3)
Face	A Samsung Galaxy S10 behind the face plate. Its screen is the eyes, its speaker is the voice. A cable keeps it charged	—
Brain	A Raspberry Pi 5 beside the base. It runs the robot's software and talks to the motors through a USB adapter (the U2D2)	—

Seventeen motors on one cable. Every motor is daisy-chained on a single bus, and every motor has an ID number. The ID is how the software knows which motor is which, so setting the IDs is the first thing you do with the motors (Stage 2).

The Pi is your workbench computer. From Stage 2 on, every command in this book is typed on your robot's own Pi, with a monitor, keyboard and mouse plugged into it. You don't need a laptop. The only other computer you'll use is for Blender, if you design a head in Stage 6.

The six stages

Stage	What	Who	You're done when
1 · Print	Test prints, then every part	Print lead	Every part is printed, checked and labelled
2 · Motor IDs	Set up the Pi; give each motor its ID	Motors lead + software lead + mentor	A scan finds all 17 motors
3 · Assembly	Head, antennae, face, arms, base, power	Mechanical lead + motors lead + mentor	The robot is built and its head's "level" is recorded

Stage	What	Who	You're done when
4 • Software	Name, calibration, arms, autostart, phone	Software lead + mentor	The robot starts by itself, calibrated
5 • Testing	Acceptance test, Code Lab, VR	Whole team	Every box on the checklist is ticked
6 • Make it yours	Optional: phone, voice, eyes, your own head	Designer + everyone	Whenever you like

Every stage ends in a  **checkpoint**. Don't start the next stage until your robot passes it, and don't start a stage at all until the pilot robot (R1) has passed it. Where a step says **mentor**, a mentor is at the table for it.

How six teams build six robots

Every robot has a number, R1 to R6, for its whole life. Put it on everything: printed parts (pencil or marker on a hidden face), motors (painter's tape), cables, the SD card, the phone, the USB stick, the parts bin. Six robots' worth of identical parts on one table is how a calibrated part ends up in the wrong robot.

R1 is the pilot robot. One team runs each stage a few days ahead of everyone else. The other five start a stage only after R1 has passed its checkpoint. A problem gets found once, not six times. The same goes for printing: print one of anything new, check it, then print five more.

Parts that belong to one robot never move to another. Motors (their IDs and positions are recorded), rods, motor arms, the calibration file and the phone all get matched to one robot. Swapping a calibrated part is how the robot on the next table starts moving wrong.

Roles on a team (4–5 people)

Role	Owns
Print lead	The team's print queue, slicer settings, labelling printed parts
Motors & wiring lead	Motor IDs, cables, the bus, strain relief, the power checks
Mechanical lead	Rods, motor arms, the head and neck, the arms and belts, the base
Software lead	The Pi, the phone, the robot's name and voice (with the mentor)
Designer	Optional: the robot's own head in Blender (everyone can help)

Everyone owns safety.

Safety — non-negotiable

1. **The motors are 5-volt parts.** The robot's power brick is 12 V, and a converter turns that into 5 V for the motors. 12 V on the motor bus destroys **every motor at once**. Only a mentor wires or changes the power parts, and the 5 V is measured with a meter before any motor is connected.

2. **Power off before you plug or unplug any motor cable.** Hot-plugging is how a motor gets a wrong ID or drops off the bus.
3. **Keep fingers out of the neck and the belts while the motors are powered.** Six motor arms and six rods make a pinch point, and so does every belt. Before you move the head or an arm by hand, it must be *limp* (torque off).
4. **An arm falls when it goes limp.** Hold it, or make sure nothing is under it, before you switch the motors off or press PANIC.
5. **Soldering only with a mentor present.** Eye protection, ventilation, iron in its stand.
6. **Hot water for the ball joints (80–90 °C)** — tongs or gloves, mentor present.
7. **Phones:** never put a phone with a swollen battery into the head. The phone charges inside the head, so check its back is still flat every time the head is opened.
8. **Printers:** don't touch the nozzle or the bed. PETG needs ventilation.
9. **VR:** the person in the headset can't see the real robot. Someone else always watches the robot, with a hand near the power switch.
10. **Children will touch this robot.** Nothing sharp: round every point and edge, including the ends of the antenna wire.

Day one

1. **Form six teams**, give each a robot number (R1 = the pilot team) and assign roles.
2. **Safety briefing** — the list above, out loud.
3. **Start printing the test pieces** (Stage 1). They answer "will it fit?" before anyone commits a ten-hour print. Also start the long, safe prints: `body_top`, `body_down`, `head_back`, `base_body`.
4. **Inventory what has arrived** against the [shopping list](#). Label boxes R1–R6.
5. **Software leads:** set up your Pi (Stage 2, step 1). It can happen while the printers run.
6. **Designers:** if your team wants its own head, install Blender and sketch the character on paper first ([Blender](#)). Design runs alongside the build.

Where the files are

Everything to print, and this book, is in the **ollie-3d** repository. Your team also gets a **USB stick** from the mentor with the same files plus the robot software:

Folder	What's in it	Where
<code>print/</code>	Every part for one robot, grouped in printing order. All millimetres — print at 100 % . <code>PRINT_LIST.csv</code> says how many of each and how to lay it on the bed	ollie-3d and the USB stick
<code>blender/</code>	The Blender starter file and its check script (Stage 6)	ollie-3d and the USB stick
<code>Reachy-Student-Build-Book.pdf</code>	This book, ready to print	ollie-3d and the USB stick
<code>software/</code>	<code>reachy-software.tar.gz</code> — the robot software, installed onto the Pi in Stage 2	USB stick only

Keep the stick: it's also where your robot's calibration file is backed up.

Credits. The stock Reachy Mini parts — the head back, crown cover, neck and Stewart plates, antenna parts and body shells — are Pollen Robotics' designs, converted here to millimetres. Everything else was designed for Ollie.

Stage 1 — Print everything

Everything gets printed before anything gets built. Printing takes days and the build takes hours, and a part that doesn't fit is far cheaper to find now than halfway through the assembly. The print lead owns this stage; everyone else can help sort, clean up and label parts as they come off the printer.

Use the files in the `print/` folder — in the **ollie-3d** repository, or on your team's USB stick. They are already in millimetres and already turned the right way up, so **print everything at 100 %, and don't rotate the arm parts.** (If you take a file straight from the robot's main repository instead: the stock Reachy parts in `hardware/oem/` are modelled in metres and import 1000× too small.)

Default settings unless a row says otherwise: **PETG, 0.2 mm layers, 3 walls, 15–20 % infill.**

1. Test prints — before anything else

Each of these is small and answers one question that would otherwise cost a big print. R1 prints them first; the other teams print them once R1's have passed. Write every result in the team log.

Print	File	Question it answers	Pass when
Hug test	<code>eye_hug_test.stl</code>	How much grip does <i>your</i> printer and plastic give a ball joint?	A ball pressed into the longest-stub eye can't be pulled off by hand
Balls	<code>balls_production_9.05_x6.stl</code>	Are the balls the right size?	Calipers read 9.00–9.10 mm . At 8.95 the joint holds half as well
One rod	<code>rod_v6_eye_rod_122mm.stl</code>	Does the snap ring hold a ball?	Ball swaged in warm (Stage 3), swivels freely, won't pull out
One motor arm	<code>arm_30mm.stl</code>	Does the arm fit the motor and the rod?	Sits flat on the motor's round boss, 4 × M2 screws bite, the rod bolts on
Yaw clamp	<code>yaw_clamp.stl</code>	Does the base's hardest joint fit the body-turn motor?	Its nose seats on the XC330's boss with no rocking. Print it nose up
Face plate	<code>s10_face_plate.stl</code> + <code>s10_face_cap.stl</code>	Does <i>your</i> S10 fit?	Test it the way it goes in the head — charging port at the service-cap end, cable plugged in . The phone slides in with a firm push at the end, the cap screws shut over the cable, nothing rattles or rocks
Arm joint coupon	<code>coupon_joints.stl</code>	Do the arm's two printed-in-place joints come out loose enough to turn?	After breaking them loose by hand, both joints turn with a fingertip. This is the most important test print — the first arm came off the bed with its elbow welded solid

Print	File	Question it answers	Pass when
Hand snap	hand_adapter.stl + hand.stl	Does the hand's snap fit your printer?	The hand pushes onto the adapter and clicks; it pulls off only with effort. An over-squashed first layer makes the snap too tight, and a snap that won't flex splits the hand

If a test fails, **stop and tell the mentor** before printing the real parts. A fit is the one thing only your printer can answer.

2. Print list — one robot

print/PRINT_LIST.csv is the same list, with a *done* column to tick off.

Head and neck

Part	File	Qty	Notes
Head back	head_back.stl	1	Supports under the openings
Crown cover	head_mic.stl	1	Easy to forget — without it the top of the head is an open slot
Face plate	s10_face_plate.stl	1	Face down on the bed, no supports. Sized for a Galaxy S10 only
Service cap	s10_face_cap.stl	1	Tongue side down, no supports
Neck plate	neck_reference.stl	1	20 % infill. Its six numbered holes take the rod tops
Stewart main plate	stewart_main_plate.stl	1	20 % infill
Stewart tricap	stewart_tricap.stl	1	20 % infill
Rods	rod_v6_eye_rod_122mm.stl	6 (+2 spare)	0.15 mm layers , no supports, lying flat, 4 walls, 40 %+ infill. It must be the v6_eye file — see below
Balls	balls_production_9.05_x6.stl	2 plates (12 balls)	As oriented (bore down), 0.10–0.15 mm layers. Print a third plate for spares
Motor arms	arm_30mm.stl	6 (+2 spare)	Flat, 4 walls, 50 %+ infill — these carry full motor torque

⚠ The rod file is rod_v6_eye_rod_122mm.stl and nothing else. The robot's main repository also holds older lengths and other joint styles, and one of them has been printed by mistake before. The right one has a raised collar around the ball hole on **both** faces.

Antennae

Part	File	Qty	Notes
Antenna hub	antenna_body.stl	2	One per antenna
Antenna interface	antenna_interface.stl	2	One per antenna
Motor holder, left	antenna_holder_l.stl	1	20 % infill
Motor holder, right	antenna_holder_r.stl	1	20 % infill

The antennae themselves aren't printed: they're stiff wire, bent by hand in Stage 3.

Body

Part	File	Qty	Notes
Body top	body_top.stl	1	Supports under the openings. Long print — start early. The arms screw to it
Body bottom	body_down.stl	1	Supports under the openings

Base, yaw joint and Pi case

Part	File	Qty	Notes
Plinth	base_body.stl	1	About ten hours. Upright, no supports, 20 % infill. Turn on bridge detection (the window ceiling is a 62 mm bridge)
Stator (cap)	base_top.stl	1	Flat, upright, no supports, 25–30 % infill — carries the robot
Rotor	yaw_rotor.stl	1	Flat, upright, no supports, 25–30 % infill
Clamp / drive hub	yaw_clamp.stl	1	Print it NOSE UP — flat on the bed puts the nose under the bed. Keep your test print if it passed
U2D2 tray	u2d2_tray.stl	1	No supports
Shelf (optional)	base_shelf.stl	1	Disc down, legs up, no supports
Pi case tray	pi5_tray.stl	1	As it loads, no supports, 20 %
Pi case cover	pi5_cover.stl	1	Upside down — roof flat on the bed

Arms — both arms

Every arm file in the kit is **already the right way up**. The two `arm_printed` parts are the exception to every other habit you have: **don't rotate them on the bed**, because their joints are printed in place and the gaps in them are sized for exactly that orientation.

Part	File	Qty	Notes
Arm, left	arm_printed_left.stl	1	The shoulder fork, upper arm and forearm in one print, with both joints printed in place . As loaded, don't rotate. Support on build plate only
Arm, right	arm_printed_right.stl	1	The mirror image — not the same part. Same rules

Part	File	Qty	Notes
Shoulder housing	shoulder_housing.stl	2	As loaded, don't rotate. It screws to the body
Hand	hand.stl	2	Serves both arms
Hand adapter	hand_adapter.stl	2	The hand snaps onto it
Wrist clamp	wrist_clamp.stl	2	Holds the wrist motor in its cradle
Boss adapter	boss_adapter.stl	6	Printed on its side, as loaded
Small pulley, 20 teeth	pulley_20t.stl	6	On each belted motor
Big pulley, 40 teeth	pulley_40t_flange.stl	6	On each joint
Roll belt cover, left	belt_cover_roll_left.stl	1	Flat, no supports
Roll belt cover, right	belt_cover_roll_right.stl	1	Flat, no supports. The mirror of the left
Elbow belt cover	belt_cover_elbow.stl	2	Flat, no supports. Serves both arms
Bushing	bushing.stl	4	Two per shoulder housing
Pitch shaft	shaft_pitch_44mm.stl	2	Lying down — stood up, it snaps along its layers
Shaft collars	collar_2mm.stl, collar_3mm.stl, collar_4mm.stl	2 each	Shims for the pitch shaft. Print all three sizes; Stage 3 uses whichever fits

Three rules for the two `arm_printed` parts, all learned from arms that came off the bed stuck:

- **Support on build plate only.** A slicer allowed to support *everywhere* will try to fill the 0.4–0.7 mm gaps inside the joints, and a joint with support inside it never moves.
- **If the slicer offers to "repair", "merge" or "make solid", say no.** Each file is six separate pieces on purpose — the gaps between them *are* the joints.
- **Don't over-squash the first layer.** It eats the joint's clearance on its own.

Weigh the first `arm_printed` as it comes off: about 62 g. If it's far off that, tell the mentor — the arm's strength numbers assume it.

Do not print

- `head_front.stl` — replaced by the S10 face plate.
- `body_foot.stl` and `body_turning.stl` — replaced by the printed base and yaw joint.
- `pitch_fork.stl`, `upper_arm.stl`, `forearm.stl` — these are the arm's parts drawn separately for the 3D preview. They can never be joined; `arm_printed` is the file.
- `shoulder_spine.stl` — it ties the two shoulders together for an arm built on its own. On this robot the shoulders screw straight to the body shell instead.

- Anything in the main repository's `hardware/oem/reference/` — those are models of parts you **buy**.

None of these are in `print/`, which is the easiest way to avoid printing them.

How much plastic

One robot is about **1.5 litres of solid plastic** — 1.2 litres for the head, body and base and 0.35 for the two arms (measured off the files). At the infills above that prints at roughly **1.3 kg of PETG per robot**. Test pieces, spares and reprints add about half as much again, so **order about 12 kg of PETG for six robots**.

3. Label everything

As each part comes off the printer: clean off supports and brims, then write the robot number on a hidden face in pencil or marker, and put it in the robot's bin. Left and right parts go in separate bags marked **L** and **R** — the left and right arm parts look alike and are not the same.

✓ Checkpoint 1

- ☐ Every test print passed, and the results are in the team log
- ☐ Every part on the print list is printed and ticked off in `PRINT_LIST.csv`
- ☐ Both `arm_printed` parts weighed, and their joints not yet forced (Stage 3 breaks them loose)
- ☐ Every part labelled with the robot number; left and right arm parts bagged separately

Stage 2 — Motor IDs

Every DYNAMIXEL motor leaves the factory as **ID 1 at 57600 baud**. Seventeen motors all answering to "1" on one cable just talk over each other, so each motor gets its own ID **before** they are chained together — one motor at a time, with nothing mechanical attached. The tool that does it runs on your robot's Pi, so the Pi is set up first.

1. Set up the Pi — your workbench computer

Software lead, with the mentor. This can happen on day one while Stage 1 prints.

You need: the Raspberry Pi 5 and its **official 27 W power supply**, the Active Cooler, the SD card the mentor prepared for your robot, a monitor and a micro-HDMI cable, a USB keyboard and mouse, your team's USB stick, and the school wifi (the install downloads packages).

1. Press the **Active Cooler** onto the Pi (its two push-pins go into the holes beside the processor) and plug its fan lead into the socket marked FAN.
2. SD card in, then monitor (the HDMI port nearest the power socket), keyboard and mouse, and power last. The Pi starts to the desktop.
3. Connect to the wifi from the network icon at the top right, if the mentor hasn't already.
4. Plug in the USB stick. In the file manager, copy `software/reachy-software.tar.gz` from the stick into your **home folder**.
5. Open a **Terminal** (the black icon on the top bar) and install the software:

```
tar -xzf reachy-software.tar.gz
cd ~/reachy
bash scripts/provision_pi.sh
```

It takes several minutes and ends with **Done**. It installs the robot software, the motor tools, and the permission to use the motor adapter.

6. **Restart the Pi** (`sudo reboot`). That permission only takes effect after a restart.

From here on, every command in this book is typed in a Terminal on the Pi, starting in the `reachy` folder.

A new Terminal starts in your home folder, so type `cd ~/reachy` first.

2. Motor power on the bench (mentor)

The motors take **5 V** from the **U2D2 Power Hub**. On this robot the Power Hub is fed by a converter that turns the 12 V power brick into 5 V ([Appendix B](#) says how the mentor wires it).

1. **Mentor:** with no motor plugged in, switch on and measure the Power Hub's input with a meter. It must read **about 5.0 V** (4.9–5.1). If it reads 12, stop: something is wired wrong, and 12 V destroys every motor it reaches.
2. Switch the Power Hub **off**. Plug the U2D2 into one of the Pi's USB ports, and connect the U2D2 to the Power Hub with the short 3-pin cable.

3. The ID map — the same on every robot

ID	Motor	Model number the scripts print	Where it goes
1	XL330- M077	1190	Left antenna (the robot's own left)
2	XL330- M077	1190	Right antenna
3	XC330 -M288 (metal case)	1240	Body turn, in the base
4–9	XL330- M288	1200	The six head motors
14	XL330-M288	1200	Right arm — shoulder pitch (in the shoulder housing)
15	XL330-M288	1200	Right arm — shoulder roll (on the fork, above the shoulder)
16	XL330-M288	1200	Right arm — elbow (lying flat in the upper arm)
17	XL330-M288	1200	Right arm — wrist (in the forearm)
18–21	XL330-M288	1200	Left arm — the same order: pitch, roll, elbow, wrist

All seventeen run at **1 Mbps** (1 000 000 baud). IDs 10–13 are not used. The software expects exactly this map.

The arm order matters. The software reads the first ID of each arm as the shoulder pitch, the next as the roll, and so on. On Ollie two shoulder motors went in the wrong places, and the pitch slider moved the roll joint. Labelling every motor with its job now is what prevents that.

The head and the arms use the **same** motor, the XL330-M288. The two antenna motors (M077) look identical but are weaker and faster, and the body-turn motor is the one with the metal case. Check the model number the script prints every time.

4. Give each motor its ID

1. **Power Hub off.** Plug **one** motor into the Power Hub. Switch on — the motor's LED blinks once.
2. Give it its ID and the fast baud rate. For a head motor that will be ID 4:

```
cd ~/reachy
.venv/bin/python scripts/set_id.py --new-id 4 --new-baud 1000000
```

The script refuses if it sees more than one motor, which is the point. It prints the motor's model number — check it against the table.

3. **Power off.** Stick a tape label on the motor with robot, ID and job: R3 · 4 · head, R3 · 14 · R shoulder pitch, R3 · 21 · L wrist. Unplug it.
4. Repeat for all seventeen: the two **M077** motors (1190) get IDs 1 and 2, the **XC330** (1240) gets ID 3, six M288 get 4–9, and eight M288 get 14–21.

5. All together, and a health check

5. Chain all seventeen together in any order (power off while you plug), switch on, and scan:

```
.venv/bin/python scripts/scan_dynamixel.py
```

You should see **exactly seventeen** motors at 1000000: 1–2 = XL330-M077, 3 = XC330-M288, 4–9 and 14–21 = XL330-M288. **Count them.** Two motors accidentally given the same ID look exactly like one working motor, and nothing else will tell you.

6. **Does every motor actually reach where it's told?**

```
.venv/bin/python scripts/nudge_diag.py --ids 1,2,3,4,5,6,7,8,9,14,15,16,17,18,19,20,21  
.venv/bin/python scripts/motor_status.py
```

`nudge_diag` moves each motor a few degrees each way **and reads it back**. A motor whose position doesn't change while it draws only a **little** current has a broken gear — the motor spins but the output doesn't; Ollie had one straight out of the box. (A **lot** of current and no movement means something is blocking it.) `motor_status` should show **no hardware error**, about **5 V**, and room temperature on every motor.

✓ Checkpoint 2

- ☐ The Pi starts to the desktop, and `cd ~/reachy` works in a Terminal
- ☐ 5 V measured at the Power Hub before any motor was connected
- ☐ The scan shows exactly 17 motors — 1–9 and 14–21 — all at 1000000, the right model on each ID
- ☐ `nudge_diag` shows every motor reaching its targets in both directions
- ☐ Every motor labelled with robot number, ID and job

If a motor won't show up: power the motors off and on (a motor that stalled can go silent until its power is cycled), reseal its cable, try it alone, try `scan_dynamixel.py --all-bauds`. If its LED lights but it never answers, check the cable is in a 3-pin **TTL** port on the hub. Trust the ID the scan reports over the sticker — stickers get shuffled.

Stage 3 — Assembly

The Pi stays on the bench beside you for this whole stage, with its monitor and keyboard, the U2D2 plugged in, and the motors on the Power Hub. The head's motor arms are fitted while the Pi tells you exactly where each motor is, and the head's "level" is recorded on the Pi.

The stock Reachy parts (the motor cages, the neck, the body) go together as the official [Reachy Mini assembly guide](#) shows. This chapter says where our robot is different, what order to work in, and what has gone wrong before.

Step	What	Who
1	Rods, balls and motor arms	Mechanical lead
2	Head motors and neck, and recording "level"	Mechanical lead + mentor
3	Antennae	Mechanical + motors
4	The face plate	Mechanical lead
5	The arms	Mechanical lead
6	The arms onto the body	Mechanical lead + mentor
7	The base and the yaw joint	Mechanical lead
8	Power and cables	Motors lead + mentor

One program on the motors at a time. Nothing stops two programs driving the motors at once, and they fight. Run one motor command at a time, and never while the robot software is running in another Terminal.

1. Rods, balls and motor arms

The head hangs on six rods. Each rod has a ball joint at **both ends**: a printed ball snapped into a ring in the rod. Twelve balls for six rods.

Check before you start. It must be `rod_v6_eye_rod_122mm.stl` — there is a raised collar around the ball hole on **both** faces. And **measure every ball with calipers: 9.00–9.10 mm**. A ball printed small holds half as well; a ball printed big never seats. Sort out any that are off.

Swage the balls in (mentor — hot water)

The ball is a tight fit on purpose — that tightness is what keeps the rod on the head.

1. Heat water to **80–90 °C** (PETG needs the lower end).
2. Hold the rod's eye in the water for about 20 seconds (tongs).
3. Push the ball through the ring — it goes in with a firm squeeze, not a click.

4. **Work the joint round in every direction while it cools**, so the ring sets to the ball's shape instead of just gripping it.

Push each ball in **once**. Every time a ball is pulled out and pushed back the ring stretches a little. If a rod feels loose, print a new rod.

Motor arms

Clean the four small holes and the tip hole with a drill bit turned by hand. Test-fit an M3 × 18 through the tip hole and an M3 nut into its pocket on the back. **Don't screw the arms onto the motors yet** — step 2 says how, and the angle matters.

✓ Checkpoint 3.1

- ☐ 6 rods (+ spares), each with **both** balls swaged in, swivelling freely, can't be pulled off
- ☐ 6 motor arms, holes cleaned, nut fits its pocket

2. The head motors and neck

Build the six head motors (IDs 4–9) into their cages following the official guide. What is different on our robot:

Official guide	Our robot
Metal link rods	Printed <code>rod_v6</code> rods, 122 mm between ball centres
Stock motor arms	Printed <code>arm_30mm</code> arms, 4 × M2 × 5 self-tapping into the motor's round boss
Camera, lens and speaker in the head	Not fitted — the phone does all three
<code>head_front</code>	Replaced by the S10 face plate (step 4)

The six head motors sit in three pairs around the centre, **facing inward**: each motor's round output boss points at the centre column, so its arm swings in an upright plane beside the motor. At the resting pose every arm is **level**, the two arms of a pair point **away from each other**, and the rod's ball sits on the arm's inner face. Match the picture in the official guide before you tighten anything.

Which motor goes where. The software doesn't care which cage holds which ID, but all six robots use the same layout so they can be compared: **ID 4 drives the rod into neck hole 1, ID 5 into hole 2 ... ID 9 into hole 6** (the neck plate's holes are numbered). Write it in the team log.

Fitting the arms — where "level" lands matters (mentor)

A motor reports its angle as a number from 0 to 4095, and the number **jumps** from 4095 back to 0 at one point in the turn (the *seam*). If an arm's level position lands near the seam, the software thinks the arm has spun right round when it crosses it. That happened on Ollie and cost an arm removal to fix. So:

1. Motors in their cages, **no motor arms fitted**, the Power Hub on.

2. Send every motor to the middle of its range (2048):

```
cd ~/reachy
.venv/bin/python -m scripts.calibrate show
.venv/bin/python -m scripts.calibrate home
.venv/bin/python -m scripts.calibrate home
.venv/bin/python scripts/motor_status.py --ids 4,5,6,7,8,9
```

`show` must print a path ending in `reachy/data/motor_calibration.json`, then `(defaults)` — and a warning that the file is unusable. That's expected here: your robot has no calibration file yet, and that is what makes "home" mean 2048. `home` runs twice because a long move can be cut short when the script lets go. Afterwards `motor_status` should read about 2048 (± 20) on every head motor. The motors go limp but stay where they were sent.

⚠ **show first, every time.** It is the only `calibrate` command that moves nothing — every other one switches the motors on and drives them to "home" straight away.

3. Without turning the motor, fit its arm in whichever of the **four bolt positions is closest to level, pointing away from its pair partner**. It won't be exactly level — the bolt pattern only goes on in quarter turns — and that's fine.

4. Screw it down with $4 \times M2 \times 5$ (small screws in plastic — snug, not tight).

Record "level" — before any rod goes on (mentor)

This is the moment it's easy: six loose arms and a spirit level. With the rods on, the arms can't be set level one at a time.

1. Hold each head motor's arm **level** — a small spirit level, or a phone level app laid along the arm. The motors are limp; the arms stay where you put them.
2. Read all six:

```
.venv/bin/python scripts/motor_status.py --ids 4,5,6,7,8,9
```

Write the six positions (`pos`) in the team log, in ID order.

3. **Each must be between about 1000 and 3100.** Closer to 0 or 4095 than that and the arm's travel crosses the seam — take that arm off, move it one bolt position, and read it again.
4. Make your robot's calibration file from the template and put your numbers in it:

```
mkdir -p data
cp calibration-template.json data/motor_calibration.json
nano data/motor_calibration.json
```

Replace the six `2048`s after `"neutral"` with your six readings, in ID order 4–9. Leave everything else alone. **Ctrl-O** then **Enter** saves; **Ctrl-X** exits.

5. Prove the file reads:

```
.venv/bin/python -m scripts.calibrate show
```

It must say **(loaded)** and show **your** six numbers.

6. Copy `data/motor_calibration.json` onto the USB stick as `calibration-R3.json` (your robot's number). It was measured on this robot and can't be made again.

From here on, "home" sends the head to **level** instead of 2048 — which is what makes it safe to run with the rods fitted. **show must say (loaded) before any home**. If it ever says `(defaults)` — a typo in the file, a missing comma — stop: with the rods on, 2048 binds the head. (Appendix B explains the rest of the file.)

Rods

Each rod: an **M3 × 18** goes through the bottom ball into the motor arm's tip, with the nut in the pocket behind. An M3 through the top ball fixes it into its numbered hole under the neck plate.

⚠ **The one screw rule that matters.** Tighten until the ball has no wobble, then **back off a hair until it swivels freely** under finger pressure. A clamped ball joint makes the head stiff and stops it reaching its angles — on Ollie, over-tight neck screws were most of why the head "couldn't get its range".

Cables

The six head motors are daisy-chained. **Hot-glue or zip-tie every connector** so the head's movement can't pull one half out. On Ollie, a motor dropping off the bus when a cable was nudged was a regular fault until every connector had strain relief.

✓ Checkpoint 3.2

- ☐ `calibrate show` says `(loaded)` with your six level readings, all between 1000 and 3100
- ☐ The calibration file is backed up on the USB stick
- ☐ With the motors unpowered, the head moves by hand through nod, tilt and turn with no binding or clicking, and no rod pops off
- ☐ Every rod-top screw passes the "swivels freely" test
- ☐ A scan still finds all six head motors (the cables survived)

3. Antennae

The two antenna motors (**M077**, IDs 1 and 2) sit in the printed holders inside the back of the head, shafts pointing backwards. The hub goes on the motor and the wire goes in the hub.

1. Build each antenna motor into its holder (left and right holders are different parts).
2. **Plug in one antenna motor (power off), then scan.** It must appear alongside the others. On Ollie, plugging the antennae in once killed the whole bus — every motor went silent — and the cause was never found. One at a time tells you immediately which one it is.
3. Send it to the middle of its range: `calibrate show` — **must say (loaded)** — then `calibrate home` twice.

4. **Make the antenna:** stiff wire (brass or steel), bent by hand with a small coil a little above the hub — the coil is what makes it bounce. Ollie's are in the photo on the first page. Measure the hole in the antenna hub and pick wire that is a snug push fit; make it about the height of Ollie's (the stock antenna is about 165 mm). **Round off the top end** with a file or a blob of hot glue — children will touch it.
5. Fit the hub, and push the wire in so it points **straight up**. That keeps the antenna's swing clear of the motor's seam.
6. Repeat for the other antenna, then strain-relieve both cables.
7. **Record the antennae's centre:** with both wires pointing straight up, read them:

```
.venv/bin/python scripts/motor_status.py --ids 1,2
```

Open `data/motor_calibration.json` in `nano` again and put the two readings in as the `center` of `antenna_left` (ID 1) and `antenna_right` (ID 2). Set each one's `min` to its centre minus 1024 and `max` to its centre plus 1024. Save, run `calibrate show` (must say `(loaded)`), and copy the file to the USB stick again.

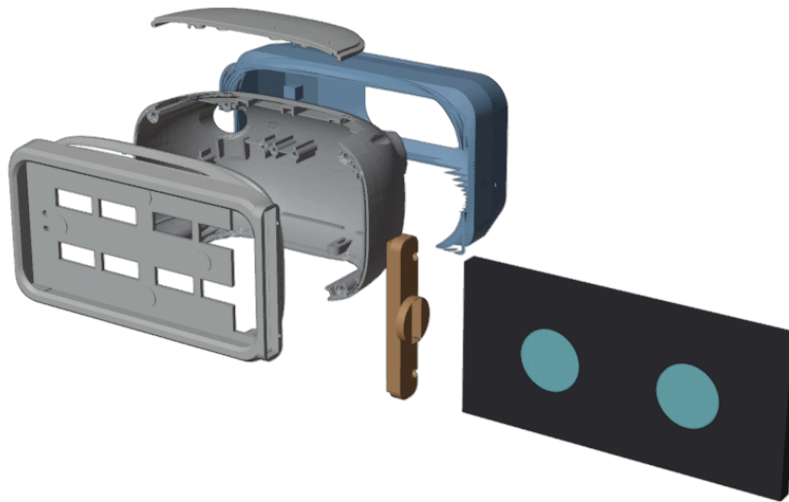
Nothing may sit in the antennae's path: they swing through a band across the top and back of the head.

✓ Checkpoint 3.3

- ☐ A scan shows 1 and 2 **and** 4–9 all at once
- ☐ Each antenna swings freely by hand (unpowered) through its whole range without hitting the head, and its tip is rounded
- ☐ The antennae's centres are in the calibration file, and it still says `(loaded)`

4. The face plate

The face is a **Samsung Galaxy S10** mounted sideways behind the face plate. Its screen draws the eyes and its speaker is the voice. The plate goes on now; **the phone goes in during Stage 4**, once it has been set up and paired — it slides in from the side, so the head doesn't have to come apart for it.



The head, pulled apart: the stock head back (grey) with the crown cover above it, the S10 face plate (white) in front, the service cap (orange) and the phone beside it — it slides in sideways — and a back shell (blue) behind. The back shell is only needed for a head designed in Stage 6.

1. Offer the face plate up to `head_back` the way the original face went on: tabs into the six slots, the locating nubs register, then the **original four screws** in from the back.
2. Clip `head_mic` into the slot across the crown.
3. Keep the service cap and its two M2 × 10 screws in the bin until Stage 4.
4. **Plan the phone's charging cable now.** Ollie's phone stays plugged in: a USB-C cable leaves the head beside the service cap and runs down behind the head into the body (you can see it in the photo on the first page). It needs enough slack for the head to nod, tilt and turn the whole way, and it must be tied where it can't catch in the neck rods. It ends at the phone's USB charger in the base (step 8).

⚠ Weight. The phone is 157 g on the front of a head held up by six small motors. Don't add anything to the head that isn't in the design.

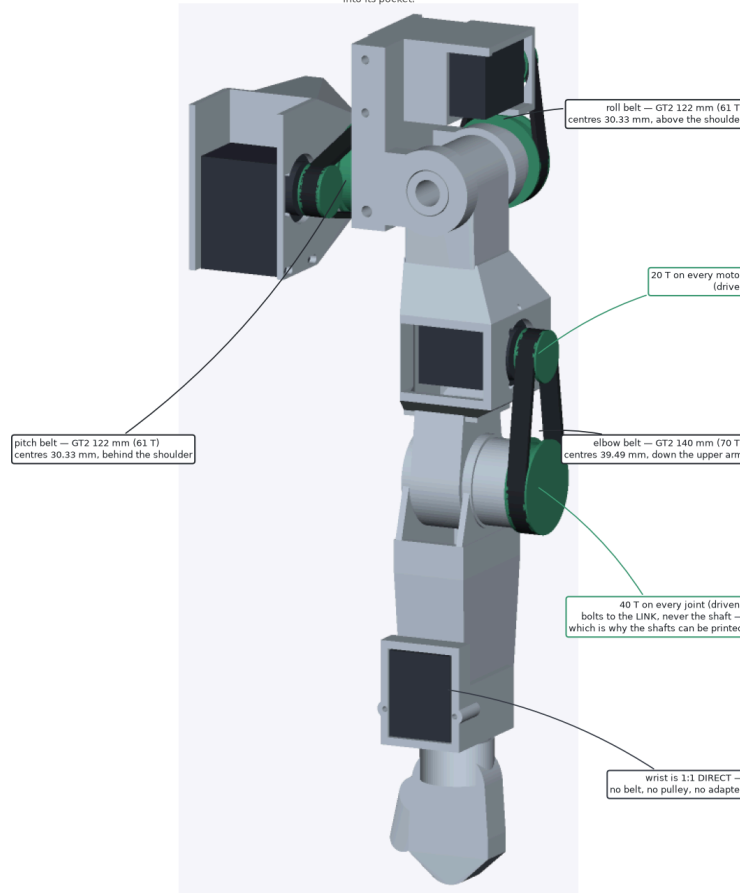
✓ Checkpoint 3.4

- ☐ Face plate on with its four screws; the top of the head is closed (`head_mic` fitted)
- ☐ The phone's cable route is planned and the cable is in the bin

5. The arms

Each arm has four motors: **shoulder pitch** (swings the arm forward and back), **shoulder roll** (lifts it out to the side), **elbow**, and **wrist**. The first three drive their joints through belts at 2:1; the wrist drives the hand directly.

Where the three belts run — 2:1 at every load-bearing axis
Links ghosted. Every belt is a stock closed loop: no tensioner, so it goes on before its motor drops into its pocket.



One arm, with the belts shown: the pitch belt behind the shoulder, the roll belt above it, the elbow belt down the upper arm. Every belt is a closed loop with no tensioner, so it goes on before its motor goes into its pocket.

Before you start:

- **Sort the parts into two piles, left and right.** `arm_printed`, the roll belt cover and the motors (14–17 right, 18–21 left) are different for each side. On a finished arm, whichever side it is, **the elbow belt runs down the outboard face of the upper arm, and the roll pulley faces forward.** If your dry fit puts them the other way, you have the other arm's part.
- **Which motor goes where** — check the labels from Stage 2 as each one goes in:

Joint	Right arm	Left arm	Where it sits
Shoulder pitch	14	18	In the shoulder housing
Shoulder roll	15	19	On the fork, above the shoulder
Elbow	16	20	Lying flat in the upper arm
Wrist	17	21	In the forearm's cradle

- **The belts** (per arm): two **122 mm** GT2 loops (pitch and roll — the same belt, so nothing to mix up) and one **140 mm** loop (elbow), all 6 mm wide.

- **No need to position the arm motors first.** Unlike the head, the arm's zero is recorded after the arm is built (Stage 4), so the motors can be at any angle when the belts go on.

Work from the hand inward. A closed belt has no slack, so each one is looped over both its pulleys while the motor is loose in your hand, and only then does the motor slide into its pocket. Every step below keeps the screwdriver's path open for the next.

Step 1 — Break the printed joints loose

The `arm_printed` part came off the printer with its elbow and shoulder joints already made. Straight off the bed the gaps may be lightly stuck together. **Work the elbow first, gripping the forearm close to the joint** — a long lever on a stuck joint snaps a 3 mm cheek off instead of freeing it, and that is exactly how the first arm was lost. Then work the shoulder. Both should turn by fingertip. **If either won't move at all, stop** and tell the mentor — don't force it.

Step 2 — Bushings and the pitch pulley

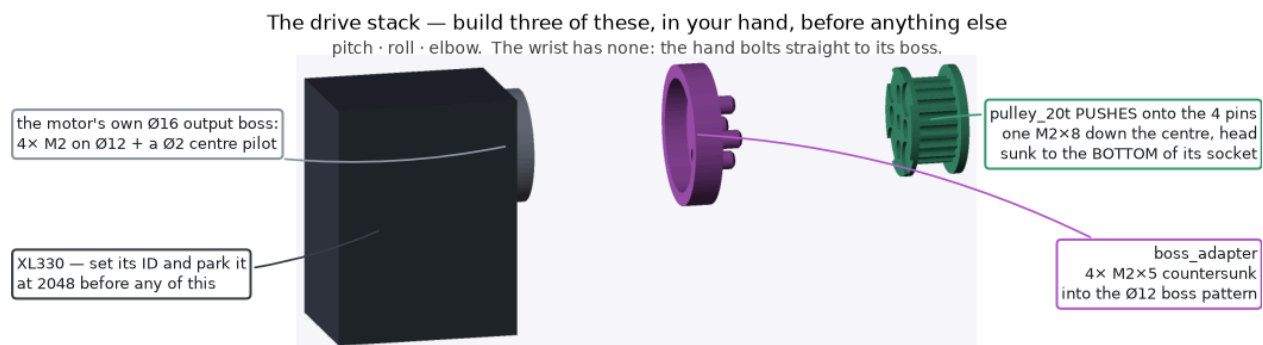
1. Press the **two bushings** into the shoulder housing, one from each end face of its bearing barrel. Press them square in a vice with a flat plate. **Test the first one:** it should need a firm push and then stay put, and the printed shaft should turn freely in it afterwards.
2. **Bolt the pitch 40-tooth pulley onto the fork now** — its inboard face (the side that will face the body), 3 × M2 × 16 straight into the printed hub. Once the arm is in the housing those screws can't be reached. Snug, not tight: you're cutting a thread in plastic. The pulley's flange must sit flat on the hub face with no gap.

Step 3 — Drive stacks (pitch, roll and elbow motors)

For each of the three belted motors, in your hand:

1. `boss_adapter` over the motor's round boss, 4 × M2 × 5 **countersunk**. Each head ends up flush or just below the plate's top face. Snug only.
2. `pulley_20t` drops onto the adapter's four drive pins.
3. One **M2 × 8** down the centre. Its head sits at the **bottom** of the pulley's socket, not flush at the top — the space above it is where a belt cover's pin goes.

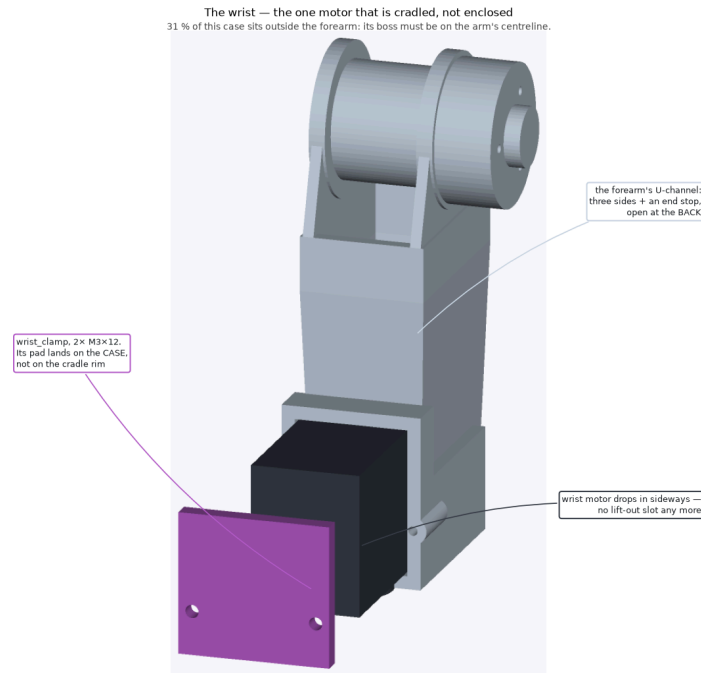
Spin it and watch the flange: any wobble means the adapter isn't seated on the boss.



The drive stack: motor, boss adapter, 20-tooth pulley, centre screw.

Step 4 — Hand and wrist

1. Drop the wrist motor (17 / 21) into the forearm's cradle from the back, and bolt the `wrist_clamp` across the opening with $2 \times M3 \times 12$.
2. `hand_adapter` onto the wrist motor's boss — $4 \times M2 \times 5$ countersunk, **while you can still reach them**.
3. Push the hand onto the adapter until the snap clicks. No screws in the hand.

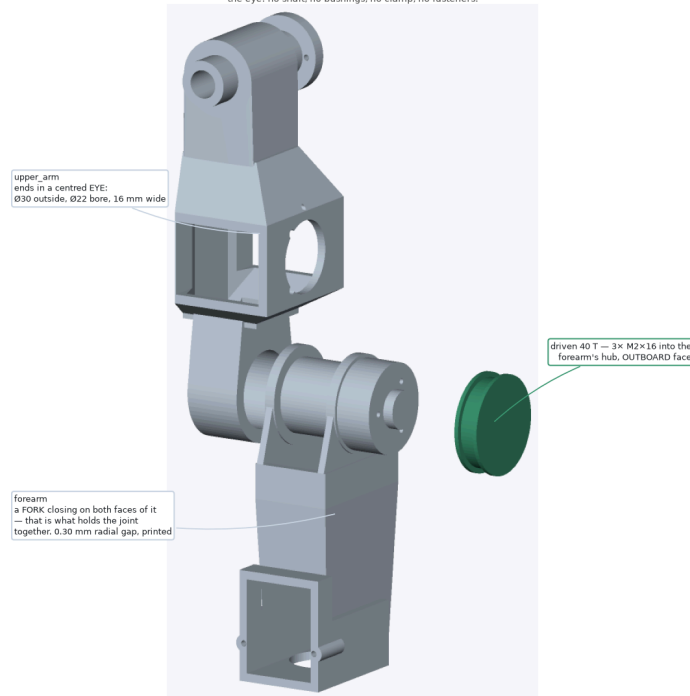


The wrist motor drops into the forearm's cradle from the back and the clamp bar holds it.

Step 5 — Elbow

1. Loop the **140 mm** belt over a 40-tooth pulley and over the elbow motor's 20-tooth pulley, motor still loose.
2. Bolt that 40-tooth pulley to the forearm's elbow hub — $3 \times M2 \times 16$, on the **outboard** face — with the belt still around it.
3. Slide the elbow motor (16 / 20) into the pocket in the upper arm **from the back** — it lies flat, long side front to back. Fix it with $2 \times M2 \times 6$ and washers through the front face.

Elbow — PRINTED IN PLACE, shown separated only to explain it
 There is no elbow assembly step. The upper arm and forearm come off the bed already joined — print arm_printed.stl, one job — and the only thing you fit here is the pulley. The fork's own cheeks capture the eye: no shaft, no bushings, no clamp, no fasteners.

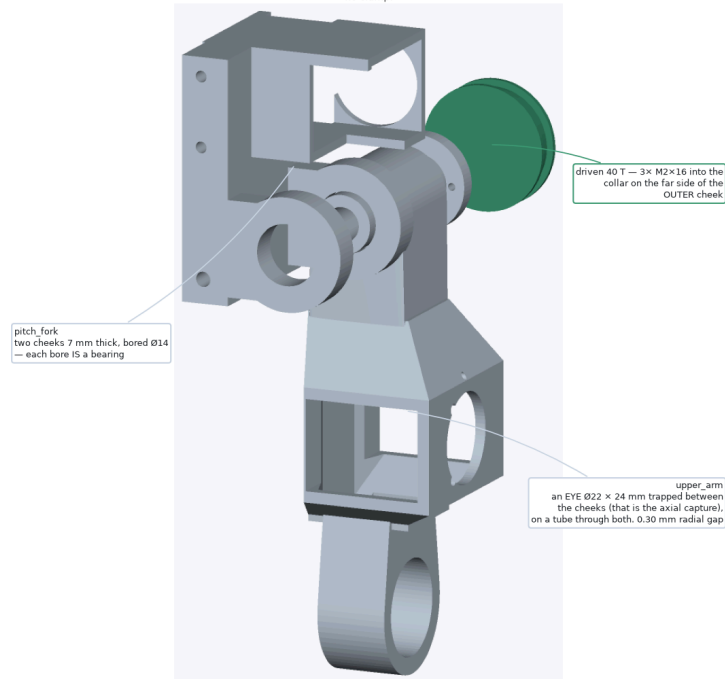


The elbow: the belt goes on first, then the motor slides in from the back.

Step 6 — Shoulder roll

1. Loop a **122 mm** belt over the roll motor's 20-tooth pulley, and over a 40-tooth pulley.
2. Bolt that 40-tooth pulley to the collar on the **far side of the fork's outer cheek** — 3 × M2 × 16. It drops onto a round spigot and should go on with a firm push and no rock.
3. Drop the roll motor (15 / 19) onto the fork's cradle from the outside, and fix it with 2 × M2 × 20 through the back face.

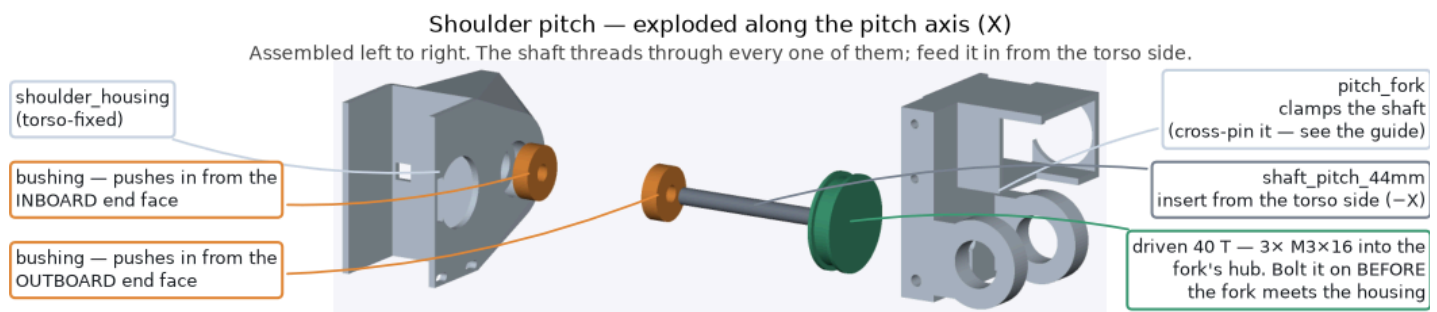
Shoulder roll — PRINTED IN PLACE, shown separated only to explain it
 There is no roll assembly step either. The fork and the upper arm come off the bed already joined — print arm, printed.stl, one job — and the only thing you fit here is the pulley. It STRADDLES: a cheek each side, so the arm hangs between its journals rather than off the end of one. No shaft, no bushings, no clamp.



Shoulder roll: the belt over both pulleys, then the motor down onto the fork's cradle.

Step 7 — Shoulder pitch

1. Loop a **122 mm** belt over the fork's pitch pulley (fitted in step 2) and the pitch motor's 20-tooth pulley.
2. Slide `shaft_pitch_44mm` in **from the body side**: through both housing bushings, through the pulley, and into the fork's pitch hub. Fit whichever collar (2, 3 or 4 mm) stops the shaft sliding end to end.
3. **Pin the shaft to the fork** (mentor): drill **2.5 mm straight up from the underside of the fork's back plate**, through the hub and the shaft together, and drive in an M3 self-tapping screw or a 2.5 mm pin. This is the only thing that stops the shaft walking out. Don't skip it.
4. Lower the pitch motor (14 / 18) into the housing's bay **from above**. Start its two M2 × 6 screws in the lower pair of slots, finger tight.
5. **Tension the belt**: put a small flat screwdriver through the little window in the front of the bay and lever the motor backwards against the belt, then tighten the two screws while you hold it there. A fraction of a millimetre is normal.



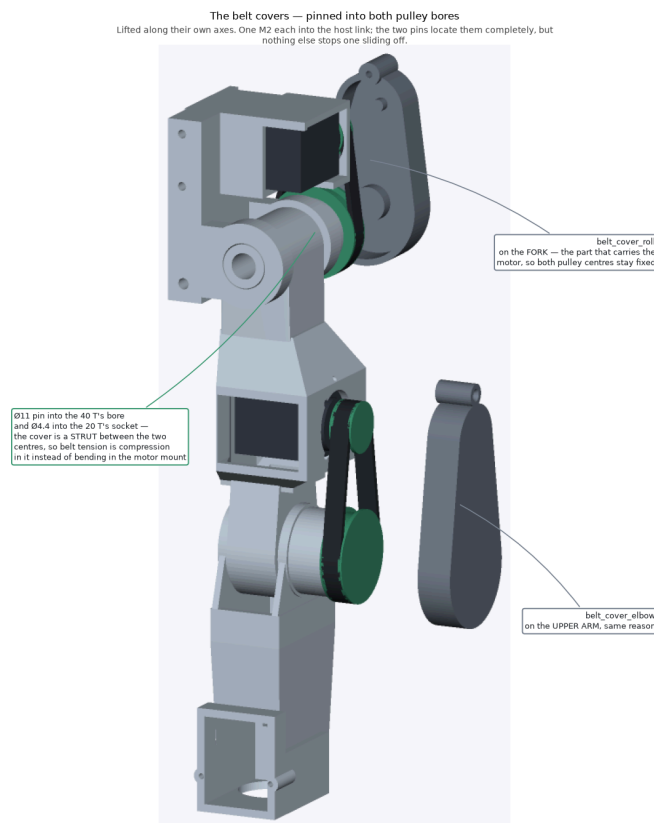
Shoulder pitch: the shaft goes in from the body side; the motor drops into the housing from above. (The picture's label says M3 × 16 for the big pulley — that's out of date: it's M2 × 16, as in step 2.)

Step 8 — Belt covers

Last, once both belts are on and both motors are fixed. Each cover pins into the bore of both its pulleys and is held by one **M2 × 16 countersunk** screw.

1. Line the big pin up with the 40-tooth pulley's bore and the small pin with the 20-tooth pulley's socket, and drop it on. **Both pins are loose, and that's right** — each pulley keeps turning under its pin. Never force a cover on.
2. Drive the screw through the cover into the fork (roll) or the upper arm (elbow). **Nip it up, don't crank it.**
3. **Spin each pulley with a fingertip before and after the screw goes in.** Stiff before means a pin is too tight in its bore; free before and stiff after means the screw is too tight.

The pitch belt has no cover — it's walled in by the body.



The two belt covers, lifted off. Each one is also a strut that holds its two pulleys apart.

Belt tension, all three: a belt should give a couple of millimetres under a firm thumb in the middle and make a dull thud, not a note. Too tight wears out the motor's bearings.

✓ Checkpoint 3.5

- ☐ Both arms built, every motor in the right place for its label
- ☐ With the motors unpowered, every joint turns by hand through its range with nothing rubbing and no belt jumping teeth
- ☐ Each pulley turns under its belt cover
- ☐ The pitch shaft is pinned on both arms
- ☐ A scan finds 14–17 and 18–21

6. The arms onto the body



Ollie's shoulders: each shoulder housing is screwed to the top of the body, where the shell narrows, and the arm hangs outside it. The motor cables go in over the top.

The shoulders screw straight through the body shell. There's no printed bracket for this — Ollie's were fitted by hand — so **R1 works out the exact position first, with the mentor, and writes it down** (distance below the top rim, distance from the front seam) so all six robots match. After that, a paper template taken off R1's body lets the other teams mark and drill their shell before anything is inside it.

1. Hold each arm where Ollie's is: its shoulder housing at the top of the body where the shell narrows, one on each side and level with each other, the arm hanging outside the shell.
2. Mark the housing flange's four holes onto the shell, then drill **3.2 mm** (a small pilot hole first). Clean off the swarf, and keep it out of the head motors below.
3. Bolt each housing on with **4 × M3 screws and nuts** (R1 confirms the length — about 12 mm).

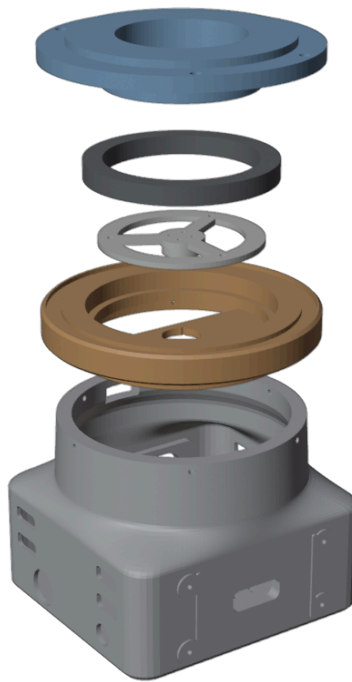
4. Run both arms' motor cables in over the top of the body to the motor chain, as in the photo. Leave enough slack for the arm's full swing, and strain-relieve every connector.

✓ Checkpoint 3.6

- ☐ Both shoulders solid on the body, at the same height, with no rocking
- ☐ With the motors unpowered, the head moves by hand through its whole range without touching either shoulder or an arm cable
- ☐ Each arm swings through its range without hitting the body
- ☐ A scan still finds 1–9 and 14–21

7. The base and yaw joint

The robot stands on a printed plinth with a **6816 bearing** on top. The body-turn motor (ID 3) hangs under a bridge in the top of the base and turns the rotor directly. Cables go straight up through the middle of the bearing.



The base, pulled apart, bottom to top: the plinth (grey), the stator cap (orange) with the yaw motor hanging under its bridge, the clamp and drive hub (white), the 6816 bearing (dark), and the rotor (blue) that the body sits on.

Do it in this order — it's the order in which every screw can still be reached:

1. Electronics into the plinth (step 8 says what goes where). The top opening is $\varnothing 117$ mm.

2. **Yaw motor** up under the cap's bridge, boss through the hole: $4 \times M2 \times 8$ down through the bridge. Send it to the middle of its range first (calibrate show must say (loaded), then calibrate home twice), and **don't turn it by hand after that**. When you choose which holes line up in steps 3 and 6, pick the combination that leaves the body **facing forward** (within about 15°). The body turns 90° each way; starting from the middle keeps that clear of the motor's seam.
3. **Clamp / hub** down onto the motor's boss: $4 \times M2 \times 10$.
4. **Bearing** into the cap's pocket.
5. **Rotor** spigot down through the bearing onto the clamp.
6. $3 \times M3 \times 8$ countersunk **up from below** at 30° , 150° and 270° , in the hole combination you chose in step 2.
7. **Four brass spacers** into the rotor's pockets, then lower the **body** onto the rotor's ring. The body's own four clamshell screws (the longer ones) go **up from under the rotor**, through the spacers, into the body — one set of screws holds the body shut and holds it on.
8. **Cap into the plinth**: $6 \times M3 \times 14$, sideways through the drum wall.
9. **Record the body's centre**: with the body facing straight forward, run `motor_status.py --ids 3` and put the reading into the file as the `center` of `body_yaw`, with `min 1536` below it and `max 1536` above it (both must stay inside 0–4095). Save, `show` must say (loaded), and back the file up to the USB stick.

To service the base later, undo the six screws in step 8 and the whole robot lifts off with the joint intact.

Cable loop. The body turns up to 90° each way, carrying the head, the arms and every cable with it. Leave enough slack in the cables passing up through the middle that a full turn each way pulls on nothing, and tie them so they can't rub on the bearing or a printed edge.

✓ Checkpoint 3.7

- ☐ With the yaw motor unpowered, the body turns smoothly by hand 90° each way with all cables connected — no rubbing, no tugging
- ☐ The body doesn't rock on the rotor
- ☐ The body's centre is in the calibration file

8. Power and cables

```

wall - 12 V brick ──┬─> 12→5 V converter ──> U2D2 Power Hub ──> all 17 motors
                    └─> 12→5 V USB charger ──> USB-C cable up through the body ──> phone

wall - official Pi 27 W supply ──> Raspberry Pi 5 ──USB──> U2D2 ──> Power Hub (data)

```

The 12 V side — the brick, the fuse, the switch and the converters — is **mentor work** and is wired and measured as [Appendix B](#) describes. The Pi keeps its own official supply: a general-purpose 5 V supply measured too low on Ollie and made the Pi crash.

1. **Mentor: measure the motor supply before any motor is connected** — probes on the Power Hub's input, power on: **about 5.0 V**. If it says 12, stop.

2. Power Hub into its tray ($4 \times M3$ from above), tray into the rails in the plinth. The converters go in the plinth too.
3. The Pi goes in its case: Active Cooler on, Pi on the standoffs ($4 \times M2.5 \times 8$), cover on. The case **stands on the table** beside the plinth and bolts to its right side ($4 \times M3 \times 16$) — the screws only locate it, the table carries it.
4. Short USB cable from the Pi to the U2D2, inside the base.
5. Motor chain from the Power Hub up through the yaw joint to the body, then on to the head and both arms.
6. The phone's charging cable from the USB charger up through the yaw joint and the body to the head (the route you planned in step 4).
7. **Tie down every cable in the base.** Nothing loose, nothing crossing the yaw joint or a printed edge where it can rub. On Ollie, the motors once lost power for eleven minutes and came back only when the cables in the base were moved — most likely a short from a loose wire.

Leave the monitor and keyboard on the Pi: you need them for Stage 4.

✓ Checkpoint 3.8

- ☐ 5 V measured at the Power Hub before the motors were connected
- ☐ With the Pi running and the motors powered, a scan from the Pi finds all 17 motors
- ☐ No loose cable anywhere in the base; the body still turns freely 90° each way

Stage 4 — Software installation

The robot software went onto the Pi in Stage 2. This stage turns it into **your** robot: its name and settings, a first start, the head's limits and the arms' zeros, starting by itself, and the phone face. Everything is done on the Pi, with its monitor and keyboard.

Step	What	Who
1	The robot's name and settings	Software lead
2	First start	Software lead + mentor
3	The head's limits, by hand-posing	Mentor + two students
4	The arms: zeros, directions, test	Mechanical lead + mentor
5	Start by itself	Software lead
6	The phone face	Software lead

One program on the motors at a time. The robot software owns the motors while it runs. If a motor goes limp under it, it switches it back on and drives it home — even if someone is holding the head. Stop the software before running **any** motor script (Ctrl-C in its Terminal now; after step 5, `sudo systemctl stop reachy`).

1. Name and settings

Give the robot a **name** — the team's choice, short, easy to say, and **not like any other robot's** (not "Ollie", and not two names that rhyme). The name is what the robot answers to, and six robots with similar names means one call wakes six.

Open the settings file:

```
cd ~/reachy
nano .env
```

- Find the line `WAKE_WORD=ollie` (**Ctrl-W**, type `WAKE_WORD`, **Enter**) and change `ollie` to your robot's name:
`WAKE_WORD=Nova`.
- Go to the very bottom of the file (**Ctrl-End**, or hold the down arrow) and add this line, which makes the voice come out of the phone in the head:

```
SPEAK=face
```

- Ctrl-O**, **Enter** to save, **Ctrl-X** to exit. Leave everything else as it is.

2. First start

1. Check the calibration file one more time: `.venv/bin/python -m scripts.calibrate show` must say **(loaded)** and show your numbers.
2. Start the robot software by hand:

```
./run.sh
```

The head rises to level and holds. **Keep a hand near the Power Hub's switch** the first time.

3. Open the web browser on the Pi and go to `http://localhost:8080` — the robot's **console**. It shows your robot and has buttons for its gestures, head poses and voice.
4. Go to `http://localhost:8080/health`. It should show `"backend": "dynamixel"`, and inside `"backend_health": "motors"` listing 1–9, and `"calibrated": true`.
5. **Ctrl-C** in the Terminal stops the robot.

3. The head's limits, by hand-posing

Mentor, with one student supporting the head and one at the keyboard.

The robot knows where "level" is. Now it learns how far the head can go in each direction. The capture tool needs the motors to itself, so the robot software must be stopped (Ctrl-C).

```
cd ~/reachy
.venv/bin/python -m scripts.calibrate show
.venv/bin/python -m scripts.calibrate capture pitch down 25
```

The head goes to level, then limp. Pose it to that limit, press **Enter**, and it saves the motor positions and goes back to level. Do all eight:

Command	Pose it to
<code>capture pitch down 25 / capture pitch up -25</code>	chin down / chin up, as far as it goes comfortably
<code>capture roll right 20 / capture roll left -20</code>	tilt to the robot's right / left
<code>capture yaw right 25 / capture yaw left -25</code>	turn to the robot's right / left
<code>capture z up 12 / capture z down -12</code>	lift the whole head straight up / lower it straight down

⚠ Support the head at the chin *and* the crown, and rotate it — don't push it. A hand on the chin pushes the whole head down while it tips, and that slide gets recorded as part of the nod: on Ollie the first nod came out as a forward head-thrust. The next command strips the slide back out, using the two `z` captures as the reference. Keep the raw file first:


```
cp data/motor_calibration.json data/motor_calibration.raw.json
python3 scripts/clean_calibration.py
python3 scripts/clean_calibration.py --write data/cleaned.json
mv data/cleaned.json data/motor_calibration.json
.venv/bin/python -m scripts.calibrate show
```

The first `clean_calibration` line only reports; the second writes the cleaned file. `show` must still say `(loaded)`. Copy **both** `motor_calibration.json` and `motor_calibration.raw.json` onto the USB stick — the raw one is how the cleaning can be redone.

Now check the directions: start the robot (`./run.sh`) and on the console press **Look R** (under *Head pose*). The head must turn to the robot's **own** right. Then drag the **Head roll** slider (on the *Manual controls* card) to the right: the head must tilt to its own right. If one is mirrored, the two captures for that axis were swapped — capture them again the other way round.

4. The arms

The arms have no "level" to record. Each joint's **zero** is captured with the arm hanging straight down, after it's built, and that's the reference every angle is measured from.

The arm goes limp when you're not driving it, and a limp arm falls. Keep hands and faces clear of it, and keep the mouse near the PANIC button. Watch the first move of every joint with a hand ready on PANIC: these arms have never been zeroed on this robot before, and a joint that has its sums wrong can try to go a long way.

1. Start the robot (`./run.sh`) and open `http://localhost:8080/limbs` on the Pi.
2. Pick the **right** side. Press  **Find motors** — it finds the arm's four motors and moves nothing. (The title at the top of the page always says "right arm"; trust the side buttons.)
3. Let the arm hang **at rest**: straight down, the forearm in line with the upper arm, the palm facing the body. Hold it still and press  **Capture zero**.
4. Tick **Drive the real arm**. The arm switches on and holds exactly where it is.
5. **Check each joint's direction**. Move each slider a little way and watch the real arm: shoulder pitch should swing the arm forward, shoulder roll should lift it out to the side, the elbow should bend the forearm forward, and the wrist should turn the hand. If a joint goes the wrong way, press its  button — that's safe while the arm is live.

If a slider moves the wrong joint — the pitch slider moves the roll joint, say — two motors are in each other's places, and  can't fix that. Untick **Drive the real arm**, pick the right motor from that joint's **id** list (picking one another joint holds swaps the two), press  **Find motors**, and check the directions again. 6. Press  **Test live robot**. It moves each joint a little, one at a time, and checks that it really moved. If it stops and lets the arm go limp, it found a joint that didn't move — a slipped belt or a stuck joint. Fix that before going on. 7. Untick **Drive the real arm**, pick the **left** side, and do steps 2–6 again. 8. **Make the arms come up with the robot**: stop the robot (Ctrl-C), open `.env` in `nano`, find `ARM_START=off` and change it to:

```
ARM_START=both
```

An arm goes limp after about 15 seconds with no command and switches back on by itself at the next one. That's normal, and it's what stops a motor holding a pose until it overheats.

5. Start by itself

When the head and both arms are calibrated, make the robot start whenever the Pi does:

```
cd ~/reachy
bash scripts/provision_pi.sh --service
sudo systemctl restart reachy
```

Restart the Pi (`sudo reboot`). About 15 seconds after the desktop appears, the head goes level on its own and `http://localhost:8080` shows the console.

From now on **the robot software owns the motors whenever the Pi is on**. Before any motor script:

```
sudo systemctl stop reachy
pgrep -af 'app\.main|app\.serve'
```

The second line must print **nothing** (if it prints anything, the software is still running). Run your script, then `sudo systemctl start reachy` afterwards.

6. The phone face

The phone is set up **before** it goes into the head — once it's inside, its settings are hard to reach.

1. **Check the battery:** the back glass must sit flat. A swollen phone never goes in a head.
2. Turn on **Stay awake:** *Settings* → *About phone* → *Software information*, tap **Build number** seven times to unlock Developer options, then *Settings* → *Developer options* → *Stay awake*.
3. Turn **auto-brightness off** and set the brightness by hand. Set the screen timeout to the maximum.
4. Put the phone on **the same wifi as the Pi**.
5. On the Pi, open `http://localhost:8080/pair?audio=1`. It shows a QR code. Scan it with the phone's camera, open the page, and in Chrome's menu (:) choose **Add to Home screen**. (The `?audio=1` matters: plain `/pair` makes the phone the eyes only, with no voice.)
6. Open the face from the new home-screen icon. **Tap it once** — phones stay silent until they're tapped. The eyes are **amber** until that tap, then they turn to normal.
7. Touch the face and press  **Test audio**. It should speak.
8. **Into the head:** slide the phone into the face plate, **screen forward, charging port at the service-cap end**. The last few millimetres take a firm push — that's the plate's pads pressing the phone onto the front lip, and it's what stops the rattle. Plug in the charging cable, lead it out past the cap the way Ollie's is, and fit the service cap: tongue in, 2 × M2 × 10 self-tapping. Don't pinch the cable.

Whenever the phone's page reloads — after the robot restarts, say — the eyes go amber again and it needs one tap before it can make a sound.

Back up: copy `data/motor_calibration.json`, `data/motor_calibration.raw.json` and `.env` onto the USB stick. (`.env` starts with a dot, so the file manager hides it — press **Ctrl-H** to show hidden files.)

✓ Checkpoint 4

- ☐ `WAKE_WORD` in `.env` is set to the robot's own name
- ☐ After a restart it comes up by itself: the head goes level and holds, `/health` shows `"backend": "dynamixel"` and, inside `"backend_health"`, motors 1–9 and `"calibrated": true`
- ☐ **Look R** turns the head to the robot's own right, and **Head roll** tilts it the right way
- ☐ Both arms passed 🎯 **Test live robot**, and `/health` shows both arms `"zeroed": true`
- ☐ The phone shows the eyes, speaks, and is charging inside the head
- ☐ The calibration files and `.env` are backed up on the USB stick

Stage 5 — Testing

Three tests, in this order: the **acceptance test** (is the robot sound?), the **Code Lab** (can you program it with blocks and JavaScript?) and **VR control** (can a headset drive it?). A robot is done when every box in this chapter is ticked.

One program on the motors at a time. The robot software owns the motors. Before running **any** motor script from the troubleshooting table: `sudo systemctl stop reachy`, check `pgrep -af 'app\.main|app\.serve'` prints nothing, and `sudo systemctl start reachy` afterwards.

Finding the robot's address. The Code Lab and VR are opened from other devices, which need the Pi's address on the wifi. In a Terminal on the Pi, `hostname -I` prints it — the first number, like `192.168.1.57`. The robot's pages are at `http://<that address>:8080`.

1. Acceptance test

The whole team, with the checklist.

- ☐ **Start-up:** unplug everything, plug it back in. The robot comes back on its own in under a minute; the head goes level; the face shows eyes (tap once for sound)
- ☐ **Gestures** from the console: 🖐️ Greet · 😊 Nod · 🗺️ Wiggle · 👁️ Look · 🎉 Celebrate · 🤔 Think — smooth, no clicking, no binding, no rod pops
- ☐ **Antennae** move, and "left" is the robot's own left
- ☐ **Body turn** — 90° each way and back, with the arms and every cable coming along cleanly
- ☐ **Arms:** after a restart, on `/limbs`, tick **Drive the real arm** and 🎯 **Test live robot** passes on both sides — without anyone capturing a zero again
- ☐ **Voice:** it says a line out of the phone; the mute switch on the console silences it
- ☐ **30-minute run**, used the way it will be at a demo: every few minutes play a few gestures, turn the body, and run a Code Lab show with the arms in it (section 2). Afterwards (software stopped) `motor_status.py` shows no hardware errors, no motor is too hot to hold a finger on, and the mentor checks the 5 V converter is warm, not hot
- ☐ Everything labelled; the team log up to date (ID map, calibration date, who did what)

2. The Code Lab — Blockly and JavaScript

The **Code Lab** is where students program the robot. You build a show out of blocks (Blockly) or write it in JavaScript, watch a cartoon of the robot act it out, and then run it on the real robot. The two are connected: **Show me the code** turns your blocks into the JavaScript that does the same thing, so you can move from blocks to typing without starting over.

The block palette isn't stored in the page — it's **built from the robot's own list** of gestures, moods, head poses and arm poses. So if the blocks appear, the page is talking to your robot. (The page calls every robot "Ollie": it's the same software.)

Open `http://<robot's address>:8080/lab` — on the Pi itself it's `http://localhost:8080/lab`, but try it from a laptop, tablet or phone on the same wifi too, because that's where students will use it.

1. **Blocks.** The  **Blocks** tab shows a palette of robot blocks. Drag in a **say** block and type "Hello", then a **do a** block and pick **celebrate** from its list. Press ► **Run**. *What Ollie will do* shows the robot's script, and the preview plays it. Open the lists on the **do a** and **look** blocks: those are your robot's own gestures and head poses.
2. **Blocks → JavaScript.** Press **Show me the code**. The  **JavaScript** tab opens with your blocks written as code, something like `ollie.say('Hello');` and `ollie.gesture('celebrate');`. Change the words in the code and press ► **Run** — the preview says the new words. That's the connection working in both directions: blocks became code, and the code runs.
3. **A loop.** Press  **Example**. It loads a program with a `for` loop that waves the antennae three times — the kind of thing blocks can't say as neatly. Press ► **Run** and watch the preview.
4. **On the real robot.** Press  **Run on Ollie**. The real robot performs the program: the voice from the phone, the antennae, the arm wave. ■ **Stop** stops it. ↺ **Step** runs one action at a time — the way to find which line does what.
5. **Send it in.** Type a name for the show and your name, and press **Submit**. It goes to the teacher's review list on the console, where a teacher can approve it.

✓ Checkpoint 5.2

- ☐ The block palette appears (the page reached the robot)
- ☐ **Show me the code** turns blocks into JavaScript, and an edited program runs in the preview
- ☐  **Run on Ollie** makes the real robot perform it, and ■ **Stop** stops it
- ☐ It works from a second device on the same wifi, not just the Pi

If it goes wrong: no blocks at all means the page couldn't load the block library or reach the robot — check the address, and tell the mentor (the software may be missing its Blockly file). If *Run on Ollie* says an arm isn't ready, that side isn't live and zeroed — go back to Stage 4, step 4.

3. VR control

With a **Meta Quest** headset you can stand next to a life-size virtual copy of your robot. Hold a **grip** button and the *real* robot's head follows your head. Hold the **trigger** and talk, and your voice comes out of the robot. The virtual robot moves along with the real one.

⚠ **Two people.** The person in the headset can't see the real robot. The second person watches the robot the whole time, with a hand near the Power Hub's switch, and says out loud what it's doing. VR never moves the arms — on purpose, for exactly this reason.

Once per robot: a certificate (mentor)

A headset only allows VR on a **secure (https)** page, so the robot needs a certificate:

```
cd ~/rechy
bash scripts/make_cert.sh
.venv/bin/pip install numpy
sudo systemctl restart reachy
```

(numpy is what lets the robot change your voice into its character voices. Without it, VR still works, and you hear your own voice.) Two things change, and both are expected:

- **The robot's pages move to `https://<address>:8080`**. Browsers warn that the certificate isn't trusted, because it's home-made. Press **Advanced → Proceed** (on the Pi, the console is now `https://localhost:8080`).
- **Plain http moves to port 8081**. The phone face uses plain http, so its home-screen icon stops working. On the phone, open `http://<address>:8081/eyes?audio=1`, tap once, choose **Add to Home screen** again, and delete the old icon.

The test

1. Put the Quest on the **same wifi as the robot**. In the Quest's browser, open `https://<address>:8080/vr` and press **Advanced → Proceed**.
2. The flat page shows your robot in 3D, moving when the real one moves.
3. Press **Enter VR. Allow the microphone** when asked — it can only ask here, before you go in.
4. The robot stands in front of you, life-size.
5. **Head**. Hold a **grip** button and turn your head to your right — the real robot turns to *its* right. Nod — it nods. Tilt — it tilts. Let go — it stops where it is. The watcher says whether each direction matched. If one is backwards, write down which, and tell the mentor (it's one setting to flip).
6. **Voice**. Hold the **trigger** and talk. When you let go, your voice — shifted towards a robot character — comes out of the phone in the robot's head. The waveform under the **Voice** button moves while you talk; if it stays flat, the microphone isn't reaching the page. Pressing **Voice** changes the character.
7. **Control panel**. On the panel beside you, the **Motion** tab has the gestures: press one and the real robot does it.
8. **Exit VR** — the button above your eyeline, apart from everything else.

✓ Checkpoint 5.3

- ☐ The virtual robot appears and follows the real one
- ☐ Holding a grip drives the real head, in the right direction on all three axes, and letting go stops it
- ☐ Your voice comes out of the robot, and the waveform moves when you talk
- ☐ A gesture from the panel plays on the real robot
- ☐ The phone face was re-added on port 8081 and still speaks

4. When something goes wrong

Stop the robot software before running any of these scripts (the box at the top of this chapter).

Symptom	Most likely	Do this
A motor missing from the scan	Cable, or it stalled and latched a fault	Power the motors off and on (not just the Pi). Reseat its cable. <code>motor_status.py --reboot <id></code> clears a latched fault
All motors vanish when one part is plugged in	A wiring fault in that part	Unplug it; test it alone with a known-good cable
A motor "moves" but the joint doesn't	Broken gear	<code>nudge_diag.py --ids <id></code> — low current and no movement = gear; high current = something blocking it
Head stiff, won't reach its angles	Neck ball-joint screws too tight	Back each one off until it swivels freely
A rod pops off a ball	Ball undersize, or a reused rod	Measure the ball; print a fresh rod
Head sags when the motors are off	Gravity — normal	Nothing
<code>calibrate show</code> says (defaults)	The calibration file is missing or has a typo	Don't run <code>home</code> . Fix the file or copy it back from the USB stick
An arm joint moves the wrong way	Its belt pulley went on the other way round	Press that joint's  on <code>/limbs</code>
A slider moves the wrong joint	Two arm motors are in each other's places	On <code>/limbs</code> , with Drive the real arm unticked, pick the right motor in that joint's <code>id</code> list, then  Find motors (Stage 4, step 4)
 Test live robot stops on one joint	A slipped belt or a stuck joint	Limp the arm, check the belt is on its teeth and the joint turns by hand
An arm clicks or jumps under load	The belt is skipping teeth	Tension it (the pitch motor has a pry window); check the cover isn't forcing a pulley
An arm is dead after a restart	<code>ARM_START</code> isn't set, or its zero is missing	<code>ARM_START=both</code> in <code>.env</code> ; <code>/health</code> → <code>arms</code> says which
The U2D2 disappears from USB	Loose USB cable	Reseat it; it comes back
The robot lists motors that aren't there	It lists what it found when it started	Restart the robot software after any rewiring
The face is silent	The phone hasn't been tapped since its page reloaded	Amber eyes = tap the face once
The phone can't open the robot's page	Not on the same wifi — or the wifi blocks devices talking to each other (many school networks do)	Check the wifi name on both; if it's the school network, ask the mentor
Code Lab: no blocks	The page didn't reach the robot, or the Blockly file is missing	Check the address; tell the mentor

Symptom	Most likely	Do this
VR: no Enter VR button, or it says VR isn't available	The page isn't on https	Use <code>https://...:8080/vr</code> (the certificate step)
VR: the head turns the wrong way	One axis is mirrored	Note which axis; the mentor flips it (Appendix B)
Body turns but stops short or creeps	A tired or damaged yaw motor	<code>nudge_diag.py --ids 3</code>

Stage 6 — Make it yours (optional)

Your robot works. Everything in this chapter is optional, and none of it should be started until your robot has passed Stage 5 — a customisation is much easier to trust on a robot you already know works.

Customisation	Effort	Who
Name, voice and eyes	Minutes, on the console	Anyone
Your own head, designed in Blender	Days — design, check, print	Designer + team
A different phone for the face	A reprinted face plate	Mentor + team

Name, voice and eyes

- **Name.** `WAKE_WORD` in `.env` (Stage 4). Change it with `nano .env`, then `sudo systemctl restart reachy`.
- **Voice.** The console's  **Speech & voice** card picks the robot's voice from a list of characters, and has sliders for how fast it talks. The robot remembers your choice after a restart.
- **Eyes.** Touch the phone's face and a bar rises from the bottom: ◀/▶ step through 36 eye styles, named as you go. Pick one before the head is closed up.
- **A more natural voice (mentor).** Out of the box the Pi speaks with *espeak*, which sounds properly robotic. For smoother voices, on the Pi:

```
cd ~/reachy
bash scripts/fetch_voices.sh
bash scripts/provision_pi.sh --voices
sudo systemctl restart reachy
```

It needs internet and about 500 MB of space.

Your own head

The next chapter, [Designing your robot's head in Blender](#), is the big one. You start from the standard head and add ears, horns, a crest, a visor or your robot's name, and a script checks your design against every rule the robot needs before it exports the file you print. Blender runs on a Windows, Mac or Linux computer — use a school computer for it, not the robot's Pi.

A different phone

The face plate is made for a **Samsung Galaxy S10** (model SM-G973) to a fraction of a millimetre: $149.9 \times 70.4 \times 7.8$ mm, 157 g. The plain S10 — not the S10e, S10+ or S10 5G. A different phone means a **new face plate**, made by the mentor from the phone's measurements.

A phone is a good candidate if:

- it's **Android with Chrome**. An iPhone can show the face, but it can't reopen the page by itself after a power cut, and a sealed head needs that;
- it's **close to the S10's size**. The S10 is already as wide as the head (150 mm), so a longer phone sticks out at the sides;
- it's **no heavier than 157 g**. Six small motors hold the head up, and the phone is most of its weight;
- it has an **OLED screen** if possible, so the black around the eyes is truly black;
- its battery is **not swollen** — the back sits flat.

Making the new plate:

1. **Measure the phone** with calipers, to 0.1 mm: length, height, thickness. Weigh it. Note where its rear camera bump is.
2. **Mentor:** put the three measurements into `PHONE_W`, `PHONE_H` and `PHONE_T` at the top of `hardware/s10-face/generate_s10_face.py` in the robot's main repository, regenerate the plate and cap, and run the fit check (`verify_s10_face.py`). The S10's camera relief (`CAM`) and screen window (`WIN_W`, `WIN_H`) are there too. The file's README says how.
3. **Print a test plate first**, exactly like the face-plate test print in Stage 1: the phone slides in with a firm push at the end, the cap screws shut, and nothing rattles.
4. **Set the eyes' size** on the new phone: touch the face, press ⚙, hold a ruler against the blue bar and adjust it until it is **exactly 50 mm**. The face then draws its eyes at the right size on that screen.
5. Set the new phone up as in Stage 4, step 6 — Stay awake, brightness, pairing, one tap.

⚠ A heavier phone changes how the head moves. After fitting one, re-run the acceptance test in Stage 5, and if the head sags or struggles to reach its limits, go back to the S10.

Designing your robot's head in Blender

Every robot gets the same neck, body and base — they hold motors, a phone and a computer, and their shapes are measured to a fraction of a millimetre. **The head is yours.** You start from the standard head and add to it: ears, horns, a crest, a visor, a badge, your robot's name engraved on its side. A script checks your design against every rule below and exports the file you print.

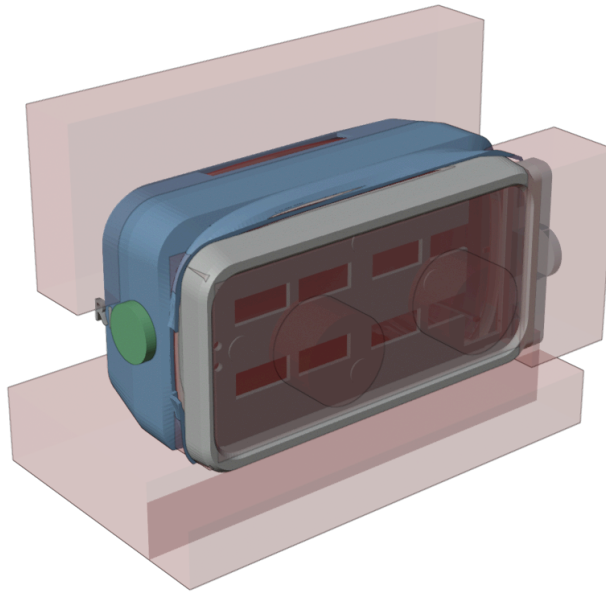


The character heads designed for the original robot — tinhead, vex, boiler, pip and iris. Every one fits the same neck and the same phone. Yours will too.

1. Set up

1. Install **Blender 4.5 LTS** from blender.org (any version from 4.2 up works). It's free and runs on Windows, Mac and Linux.
2. Optional but useful: the **3D Print Toolbox** add-on. *Edit ► Preferences ► Get Extensions*, search "3D Print Toolbox", **Install**. It checks wall thickness and overhangs.
3. Get the starter file, `reachy_head_starter.blend`, from the `blender/` folder — in the ollie-3d repository, or on your team's USB stick.
4. Open it, then straight away **File ► Save As** into your own folder, named after your robot (`R3-nova.blend`). Never work in the original.

2. What's in the starter file



Colour	What it is	Can you change it?
Blue	The back shell — the part that slides over the back of the head	No — it's locked. You add to it
White	The face plate that holds the phone	No — locked. You can add small details to it
Grey, see-through	Parts inside and underneath: the stock head back, the crown cover, the body, the service cap, the phone	No — they're there so you can see what you're designing around
Red, see-through	Keep-out zones — nothing you make may go in them	No
Green (hidden)	Two examples: an ear disc and an engraved label	Copy them, move them, delete them

The Outliner (top right) has two collections that are yours:

- **MY ADD-ONS** — every object you want to *add* to the head goes here.
- **MY CUTS** — every object you want to *carve out* of the head (engraved letters, grooves) goes here.

Anything outside those two collections is ignored when you export. To move an object into one, select it, press **M** and pick the collection.

Units are millimetres. The face looks toward **−Y**. Handy views: **Numpad 1** front, **Ctrl + Numpad 1** back, **Numpad 3** side, **Numpad 7** top. No numpad? *Edit ► Preferences ► Input ► Emulate Numpad*, then use the number row.

3. The rules

The check script tests all of these for you. Knowing them first saves you redesigns.

Rule	Why
Sink every add-on about 1 mm into the surface.	If it only touches the surface it prints as a separate piece and falls off
Stay out of the red zones.	Each one is something your design would break: the inside of the head, the phone's pocket, the eyes, the phone's glass, the service cap (the phone slides out there to charge), below the head (the neck and body), and the antenna band across the top and back, where the antennae swing
Back shell: 65 g or less in total. The plain shell weighs about 45 g, so you have about 20 g to add. Face plate: 15 g or less added.	Six small motors hold up the head and a 157 g phone. Weight on the head is the one thing they can't argue with
Size: no more than 88 mm out to each side, 75 mm up, 52 mm back, and face details no more than 5 mm proud of the face	Bigger heads hit things when they turn, and cost motor strength
Nothing below the head.	The head nods, tilts and turns right down to the body. The check swings your add-ons through every pose and fails anything that gets closer to the body than the standard head already does
Engravings no deeper than 0.8 mm.	The shell wall is only 1.6 mm thick
Don't cover the phone's camera if your design covers part of the screen.	The camera, near one end of the screen, is how the robot sees who is talking. The check can't see it — hold your print up to the phone and look
Nothing thinner than 1.2 mm. Round every point and edge — at least 1 mm radius on tips.	Thin bits snap. Children will touch this robot
Each add-on and cut is its own closed object.	Open meshes (holes, loose edges) can't be printed. Text objects are fine as they are
Nothing crosses the line where the face meets the back shell.	They are separate prints. A crest that runs from face to crown is two objects, split at that line

4. Make an add-on

The example **ear disc** shows the pattern. Turn on the green EXAMPLES collection (the eye icon in the Outliner) to see it.

1. **Add ► Mesh ► Cylinder** (or Cone, UV Sphere, Cube...).
2. Press **N** for the side panel. Type exact numbers into *Location, Rotation, Dimensions*.
3. Push it into the surface until about 1 mm is buried. Check it from two views.
4. **M ► MY ADD-ONS.**

Shaping tips:

- **Round it** with a *Bevel* modifier (width 1–2 mm, segments 3) — modifiers count; the check uses the finished shape.
- **Smooth it** with a *Subdivision Surface* modifier.

- **Horns and fins:** start from a cone, then *Edit Mode* (**Tab**) with proportional editing (**O**) to curve it.
- **Sculpting** is fine on **your own** objects. Afterwards add a *Remesh* modifier (Voxel, 0.5 mm) so it is a closed solid.
- **Mirror** left and right with a *Mirror* modifier (axis X) — both sides for the price of one.
- **Save weight:** hollow a big add-on with a *Solidify* modifier (thickness 1.6 mm), keeping it closed.

5. Engrave something

The example **engraved label** shows the pattern.

1. **Add ► Text.** **Tab** to edit it, type your robot's name, **Tab** again.
2. In the green "**a**" (Object Data) tab: *Geometry* ► *Extrude* = **1.0 mm** (that makes it 2 mm deep in total), *Size* = 10–14 mm. Chunky fonts engrave best; letters under 8 mm tall are hard to read.
3. Rotate it so its face points out of the surface, and place it so it cuts **0.8 mm** in.
4. **M ► MY CUTS.** The check shows cuts as wireframe so they don't hide what they cut.

Raised letters are the same object put in **MY ADD-ONS** instead, sunk 0.5–1 mm.

6. Check and export

1. Save (**Ctrl + S**).
2. Click the **Scripting** tab at the top. `reachy_check.py` is open in the text editor.
3. At the top of the script, set your robot and design name:

```
ROBOT = "R3"
DESIGN = "nova-horns"
KEEP_ANTENNAE = True    # False only if your robot has no antennae
```

4. Press **Run Script** (▶, or **Alt + P**). The report appears in the text editor.

A passing report looks like this:

```

REACHY HEAD CHECK - R3 / nova-horns (antennae kept)

ADD-ONS
  OK    horn_left (shell): closed, attached, clear of every keep-out zone
  OK    horn_right (shell): closed, attached, clear of every keep-out zone

CUTS
  OK    name (shell): closed, attached, clear of every keep-out zone

HEAD MOVEMENT
  OK    Swept 135 head poses: your add-ons never come closer to the body than
        the standard head does (nearest 55.5 mm, at pitch +25, roll +20, ...)

WEIGHT
  OK    back shell: 47.4 g of 65 g
  NOTE  face plate unchanged - print the standard s10_face_plate.stl

RESULT
  PASS - exported:
        .../export/R3_nova-horns_shell.stl

```

PASS means your STL files are in the **export** folder beside your `.blend`. **FAIL** means nothing was exported. Fix each line marked **FIX** and run it again:

The report says	Do this
<i>reaches inside the head</i>	Pull it outward. Only ~1 mm should be buried
<i>reaches in where the face plate slots into the shell</i>	Move it back from the front edge of the shell by a few mm
<i>covers an eye / would press on the phone's glass</i>	Move it off the eye circles; keep masks in front of the face
<i>blocks the service cap</i>	Keep the service-cap side of the face clear — the side with the small screwed-on cap (+X)
<i>the antennae swing through there</i>	Move it forward or back out of the band across the crown
<i>doesn't touch</i>	Sink it about 1 mm into the surface
<i>isn't a closed solid</i>	<i>Mesh ► Clean Up ► Fill Holes</i> , or 3D Print Toolbox ► <i>Make Manifold</i>
<i>crosses the line where the face plate meets the back shell</i>	Split it into two objects, one each side
<i>comes within ... mm of the body</i>	Make the lower part of your design smaller or higher up
<i>... g of 65 g failing</i>	Make add-ons smaller, or hollow them (Solidify)

To look at exactly what will be printed, turn on the **RESULT** collection in the Outliner. For a last check, select `PRINT_shell` and run 3D Print Toolbox ► *Check All* — thin walls and steep overhangs show up there.

7. Print it

- **Back shell:** collar-down (the open front edge flat on the bed), PETG, 3 walls, 15 % infill, 0.2 mm layers. Tall horns may want a few supports.

- **Face plate with raised details on the front:** print it **face up** (the flat rear band on the bed), with supports only inside the phone tunnel. The plain face plate prints face down, but raised details would be on the bed.
- **Weigh it** on a kitchen scale. The shell must come in at 65 g or less.
- **Colour:** paint with acrylic over a primer, or change filament at a layer height in the slicer (a colour band costs nothing in weight).

The shell slides onto the back of the head and grips by itself. No screws, so you can swap heads in seconds.

Going further

Want a completely new silhouette, not add-ons on the standard one? The character heads were generated from numbers in Python in the robot's main repository (`hardware/heads/generate_characters.py`, specified in `hardware/heads/CHARACTER_SPEC.md`), so the fit to the robot is exact. Describe your shape to your mentor. The mentor can also make a starter file from any of the character shells:

```
blender -b --factory-startup -P hardware/heads/blender/make_starter.py -- \
  --shell character_pip_shell --out pip_starter.blend
```


Appendix A — Shopping list

Everything to **buy**. What to **print** is [Stage 1](#). Quantities are **per robot**, with a **×6** column for the whole program. Prices change; confirm each listing before ordering.

Motors — the expensive part

Part	Per robot	×6	Fleet spares	Notes
DYNAMIXEL XL330-M288-T	14	84	6	6 for the head, 4 for each arm. One of the original robot's motors arrived with a mis-moulded gear — buy spares
DYNAMIXEL XL330-M077-T	2	12	1	The antennae — a weaker, faster version. Don't mix them up with the M288
DYNAMIXEL XC330-M288-T	1	6	1	The body turn. Metal case, because it turns the whole robot

All three are 5 V motors on the same 3-pin cable. Each motor comes with one short cable.

Motor bus and power

Part	Per robot	×6	Notes
ROBOTIS U2D2	1	6	The USB adapter between the Pi and the motors
ROBOTIS U2D2 Power Hub Board Set	1	6	Where the motors plug in and get their 5 V
ROBOTIS Robot Cable-X3P, assorted lengths	8	48	Longer motor cables for the arms and the run up through the body — the stock ones are short
12 V power brick, 120 W or more	1	6	The robot's one plug for the motors and the phone. Ollie's is 160 W
12 V → 5 V step-down converter, 10 A (50 W)	1	6	Feeds the Power Hub, i.e. all 17 motors. ⚠ The motors must only ever see 5 V
12 V → 5 V USB charger module (USB-A)	1	6	Charges the phone in the head
Fuse holder + 10 A fuses, rocker switch, wire, lever connectors, 2200 µF capacitor	1 set	6	The mentor builds the power wiring from these — see Appendix B

Brain — and the workbench

Part	Per robot	×6	Notes
Raspberry Pi 5, 4 GB	1	6	Ollie runs on a 2 GB board and it is a real ceiling. 4 GB removes that
Official Raspberry Pi 27 W USB-C power supply	1	6	Not a generic 5 V supply — one measured 4.99 V on Ollie and made the Pi crash
microSD, 32 GB, high-endurance (Samsung PRO Endurance or SanDisk Max Endurance)	1	6	+2 spares. Label it with the robot number
Raspberry Pi 5 Active Cooler	1	6	
USB stick, 16 GB or more	1	6	The student kit, and the robot's calibration backups
Monitor with HDMI , USB keyboard and mouse	1 set	6	The Pi is the workbench computer from Stage 2 on. Any school monitor and keyboard will do; they can be shared once a robot is finished
Micro-HDMI to HDMI cable	1	6	The Pi 5's video ports are micro-HDMI

Face

Part	Per robot	×6	Notes
Samsung Galaxy S10 (SM-G973) , used is fine	1	6	The plain S10 — not the S10e, S10+ or S10 5G. The face plate is sized to this phone to a fraction of a millimetre. Check the screen has no burn-in and the battery isn't swollen. A different phone means a new face plate (Stage 6)
USB-A to USB-C cable, about 1 m, thin	1	6	From the USB charger in the base, up through the body, to the phone

Base and yaw joint

Part	Per robot	×6	Notes
6816-2RS thin-section bearing , 80 × 100 × 10 mm	1	6	+1 spare
Brass spacer, ø6 mm OD × ø3.2 mm bore × 20 mm	4	24	R1 measures one of the body's four ø6.64 columns with a pin gauge before the fleet order
Stick-on rubber feet, ø12 mm	8	48	4 for the plinth, 4 for the Pi case

Antenna wire

Part	Per robot	×6	Notes
Stiff wire, brass or steel	about 50 cm	3 m	Bent by hand into the two antennae. R1 measures the hole in the printed antenna hub and buys wire that's a snug push fit

Arms (both arms)

Part	Per robot	×6	Notes
GT2 closed-loop belt, 122 mm (61 teeth), 6 mm wide	4	24	Shoulder pitch and shoulder roll, both arms. +1 spare per robot
GT2 closed-loop belt, 140 mm (70 teeth), 6 mm wide	2	12	The elbows. +1 spare per robot

Screws

The stock Reachy parts use the screws listed in the official [Reachy Mini assembly guide](#). Buy each team an **M2 / M2.5 / M3 assortment** (machine screws, self-tapping screws and nuts) for those. The custom parts need these specifically.

Head, face and base

Screw	Per robot	Where
M2 × 5 self-tapping	24	Motor arms → motor boss (4 per arm)
M3 × 18 + M3 nut	12 + 12	Rods: through the ball into the motor arm, and the rod tops into the neck plate. R1 confirms the top-end length
M2 × 10–12 self-tapping	2	Face service cap
M2 × 8	4	Yaw motor → the base's motor bridge
M2 × 10	4	Yaw clamp → the yaw motor's boss
M3 × 8 countersunk	3	Yaw clamp → rotor, from below
M3 × 14 thread-forming	6	Base cap → plinth, sideways through the drum wall
Body clamshell screws, longer than stock	4	They now reach through the rotor into the body. R1 measures the length
M3 × 8–10	4	U2D2 Power Hub Board → its tray
M2.5 × 8	8	Pi → tray (4), cover → tray (4)
M3 × 16 thread-forming	4	Pi case → plinth wall

Arms — for both arms

Screw	Per robot	Where
M2 × 5 countersunk (90°, DIN 965)	32	Boss adapters (4 each × 6) and hand adapters (4 each × 2). Countersunk, not pan head: the seat is a cone
M2 × 8	6	The centre screw in each small pulley
M2 × 16	18	Big pulleys → their joints, 3 each
M2 × 16 countersunk	4	One per belt cover. Not shorter — the screw crosses the cover's own boss first
M2 × 6	8	Pitch and elbow motors, 2 each
M2 washer, about 5 mm across	4	Under the elbow motors' screws
M2 × 20 self-tapping	4	Roll motors, 2 each
M3 × 12	4	Wrist clamps, 2 each
M3 × 10 self-tapping, or a 2.5 mm pin	2	Pins each pitch shaft to its fork
M3 × 12 + M3 nut	8 + 8	Shoulder housings → body shell, 4 each. R1 confirms the length

Tools

Per team: metric hex keys and small screwdrivers (Phillips #0 and #1, and a small flat blade for belt tension), digital calipers, a multimeter, a drill with 2.5 mm and 3.2 mm bits, a small vice (for pressing bushings), a small spirit level (or a phone level app), flush cutters, needle-nose pliers, a file, zip ties, painter's tape and a marker, a hot-glue gun, a deburring tool and fine sandpaper.

Shared: a kettle and a thermometer (ball joints are swaged in 80–90 °C water), a pin gauge set or drill bits to measure holes, and a Meta Quest headset for the VR test in Stage 5.

Appendix B — Mentor notes

This appendix is for the adults. It covers what has to be ready before the students start, and the parts of the build that can break hardware if they're done in the wrong order.

Before day one

1. The student kit — one USB stick per team

On a computer with the robot's main repository (`redarchlabs/reachy-the-robot`):

```
.venv-cad/bin/python docs/student-build/make_pdf.py
.venv-cad/bin/python scripts/make_student_kit.py
```

That writes `dist/reachy-student-kit/` (and a zip of it). Copy the folder onto six USB sticks labelled R1–R6. The kit holds the print files (all millimetres), this book, the Blender starter, and `software/reachy-software.tar.gz`: the robot software as committed, plus the 3D models and browser libraries (Blockly, three.js) its pages need. It deliberately holds **no** `.env`, no calibration and no voice models — a student robot starts clean.

The same print files, this book's chapters and the Blender starter are published to the **ollie-3d** repository, without the software: `.venv-cad/bin/python scripts/export_ollie3d.py <your ollie-3d checkout>`, then commit and push there.

2. The SD cards

Raspberry Pi Imager → **Raspberry Pi 5** → **Raspberry Pi OS (64-bit)**, the version with the desktop. In the Imager's settings:

- hostname `reachy-r1` ... `reachy-r6`;
- a username and password — write them down; the install in Stage 2 asks for the password;
- the wifi the robots will use, and your country, time zone and keyboard;
- SSH, if you want to reach the Pis from a laptop (optional — nothing in the book needs it).

Label each card with its robot number.

3. The network

Two things are needed from the wifi, and school networks often give only the first:

1. **Internet**, for the install in Stage 2 (system packages and Python packages).
2. **Devices that can reach each other**. The phone in the head, the Pi and (for the VR test) the headset all talk to each other directly. Many school networks block that ("client isolation"), and it shows up as a phone that simply can't open the robot's page. If so, bring a travel router and put the robots on its wifi.

Reserve each Pi's address in the router (a DHCP reservation) so it never changes — the phone's home-screen icon has the address in it.

4. Don't deploy student robots from Ollie's working copy

`scripts/deploy_pi.sh` was written for Ollie. On any Pi that doesn't have them yet, it seeds two files from the working copy it runs in: Ollie's `.env` (with its credentials) and Ollie's `data/motor_calibration.json` — motor positions measured on a different head, which can drive a student head against its own rods and break a printed rod or motor arm. Student robots are installed from the kit, on the Pi, as Stage 2 describes. If you ever do use `deploy_pi.sh` for a student Pi, run it from a fresh clone, which has neither file.

The power wiring (12 V)

```
wall - 12 V brick -[10 A fuse]-[switch]└─> 12→5 V converter, 10 A ─> U2D2 Power Hub ─> 17 motors
                                   └─> 12→5 V USB charger ─> phone, up the neck

wall - official Pi 27 W supply ─> Raspberry Pi 5    (its own supply - never the 12 V board)
```

Build it from `hardware/power/BUILD.md` in the main repository, which puts a meter reading between every step. The student robots use two of its outputs: the motor converter and the phone's USB charger. The Pi stays on its official supply — a general-purpose 5 V supply measured 4.99 V on Ollie, close enough to the Pi's cut-out that it crashed.

- Fuse the **+12 V** leg, not ground. Nothing above 12 V goes inside the base.
- **Set and measure the motor converter at 5.0 V with nothing connected**, then at the Power Hub's input, and only then connect motors. A 12 V mistake here destroys every motor on the bus.
- **Seventeen motors on one 10 A converter** is what Ollie runs. A driven arm holds itself up against gravity, so the converter works hardest while the arms are moving. Feel it after the 30-minute run in Stage 5, which includes arm shows: warm is fine; too hot to hold means the arms need a converter of their own.
- Tie every wire down. Ollie's motors once lost power for eleven minutes and came back only when the cables in the base were moved — most likely a short from a loose wire.

The calibration file

Students write it in Stage 3 from `calibration-template.json`, which is in the software:

```

{
  "joints": {
    "antenna_left": {"id": 1, "center": 2048, "ticks_per_deg": 11.4,
                     "sign": 1, "min": 1024, "max": 3072},
    "antenna_right": {"id": 2, "center": 2048, "ticks_per_deg": 11.4,
                      "sign": -1, "min": 1024, "max": 3072},
    "body_yaw": {"id": 3, "center": 2048, "ticks_per_deg": 11.4,
                 "sign": 1, "min": 512, "max": 3584}
  },
  "head": {
    "ids": [4, 5, 6, 7, 8, 9],
    "neutral": [2048, 2048, 2048, 2048, 2048, 2048],
    "presets": {},
    "jacobian": [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0],
                 [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
  }
}

```

- `neutral` is the six "level" readings, in ID order 4–9. Each must be between about 1000 and 3100, or that arm's travel crosses the motor's 4095 → 0 seam.
- Each `center` is read with the part in its middle position; `min`/`max` stay the same distance from it as above (antennae ± 1024 , body ± 1536) and inside 0–4095.
- The **all-zero jacobian is deliberate**: until the head's limits are captured in Stage 4, every head command holds level instead of guessing.
- The head's limits (Stage 4) and the arms' zeros (captured on `/limbs`) are written into the same file by the software. None of it can be regenerated — it was measured on that robot — which is why the book keeps asking for backups.

Two traps in the tools, and why the book says "`show` first" so often:

- Every `calibrate` command **except** `show` opens the motor port and drives the head to "home" before doing anything else.
- `calibrate` quietly falls back to built-in defaults when the file is missing or has a typo, and the default "home" is 2048 on every head motor — up to about 50° from level on a head with its rods fitted. So `show` must say `(loaded)` before anything else runs.

The robot software does the same when it starts: it switches the motors on and drives the head to level straight away. That's why the autostart isn't installed until Stage 4, step 5, when the calibration is complete.

The phone

- Plain http is enough for the **eyes and the voice** — no certificate.
- Phones stay silent until tapped, and that permission is lost every time the page reloads, so the eyes go amber after a restart until someone taps the face. *Fully Kiosk Browser* (Android) can remove the tap and reopen the page when the phone restarts; see `hardware/phone-eyes/README.md`.
- The phone's **microphone and camera** need https that the phone trusts (an mkcert certificate authority installed on the phone). That isn't part of this build; the top-level `README.md` of the main repository covers it.

The certificate for VR

`scripts/make_cert.sh` (Stage 5) uses `mkcert` if it's installed, and otherwise makes a self-signed certificate — that's the "Advanced → Proceed" warning. Once a certificate exists the robot serves **https on 8080 and plain http on 8081**, which is why the phone face has to be re-added on 8081. To undo it: `rm -r ~/reachy/certs` and `sudo systemctl restart reachy`, and the phone's old icon on 8080 works again.

VR head directions

VR steers the real head from the wearer's head, and the three directions are set by `YAW_SIGN`, `PITCH_SIGN` and `ROLL_SIGN` near the top of `app/static/vr.js`, which the file marks as not yet checked on hardware. If Stage 5 finds an axis backwards, flip that one constant in the main repository and rebuild the kit. It's the same code on every robot, so R1's result is everyone's.

Updating a robot's software later

Build a new kit, copy the new `reachy-software.tar.gz` into the Pi's home folder, and:

```
tar -xzf reachy-software.tar.gz
cd ~/reachy
bash scripts/provision_pi.sh
sudo systemctl restart reachy
```

The archive has no `data/` files and no `.env`, so unpacking it over the old copy leaves the robot's name, settings and calibration alone.

What still needs tooling (for whoever picks this up)

- There is no `calibrate capture-neutral`: Stage 3 reads positions with `motor_status.py` and the file is edited by hand in `nano`. A command that reads with the head limp — and never homes first — would remove the hand edit.
- `calibrate` falls back to defaults on a missing or broken file and then homes to them. Refusing to move when the file can't be read would make the `show` ritual unnecessary.
- The arms' mount to the body is a hand-fitted, drilled joint. A printed bracket or a drilling template would make six robots match without R1's measurements.